DECODED

Data Creators

Designing, Building, and Running Data Products on the Paved Path

Five-day course · Cohorts 3–5

The Week at a Glance

Day	*Theme*	*You leave with*
1	Designing data products · Catalyst & GenAI	Consumer need, product decision, first contract
2	Canvas · effective AI · contract · adoption · paved path	A canvas, a prioritised contract, tool boundaries
3	Docker · data systems · DBT	A tested dbt pipeline, built by you
4	AI agents · pipeline design · Airflow	A scheduled, audited DAG
5	Infrastructure · data operations · end-to-end	The release picture, commit to cloud

Days 1 and 2 are the design days: group challenges, whiteboarding, the product decision. Days 3 and 4 change character entirely: a 20 to 30 minute overview, then a self-paced challenge block of 45 to 60 minutes, then a regroup. Day 5 is data operations and the end-to-end picture, and it draws a line at three hours.

Day 1: Designing Data Products

DECODED

D1.1 Icebreaker: Groups, Products, Pain Points

In table groups:

- Name, role, and expectations of the programme; shared hobbies
- What data products do you work with? Brainstorm them on paper as shared examples for the next two days
- In what ways are they annoying? Or more directly: which data sets do you rely on, which don't you trust, and why?

Then a show of hands: programming experience · used DBT · used Harness.

The pain points that recur every cohort: entitlements and unclear ownership, missing SLAs, integration across too many sources, identifier misalignment (the same thing created two ways in two systems), and the liaison problem: without a person who can bridge domain and technical, a product with everything else in place still fails its consumers.

Day 1 · Morning: Understanding and Designing Data Products



D1.2 What *is* a data product?

A consumable data set? A single source of truth?

Data aligned to an enterprise standard? Something that provides business information?

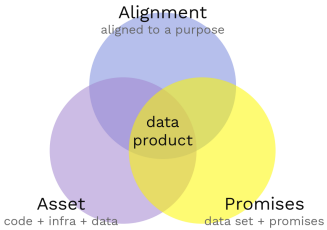
Observable? Categorisable?

Every answer here came from a cohort's brainstorm, and they are all partly right. "Single source of truth" is the questionable one; "aligned" is the useful word, and the next three slides unpack it.

D1.3 Three Views of a Data Product

1. Alignment: aligned to a purpose or goal; phrased and structured for it
2. Asset: data product = code + infra + data
3. Promises: data product = data set + promises

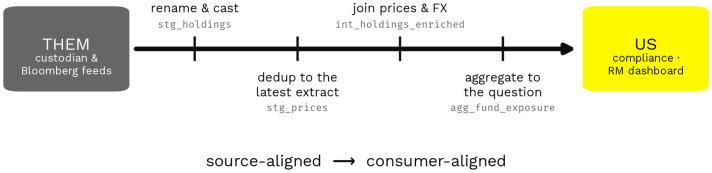
   data set + (SLAs + ownership + governance + quality + ...)



One thing, three questions: aligned to whom, made of what, promising what

The views describe one thing from different angles, and they reconcile: most promises are *automated by code* (promises meet asset), and alignment is a promise that the product will meet your needs (alignment meets promises). The promises view does the most work this week.

D1.4 Source-Aligned → Consumer-Aligned



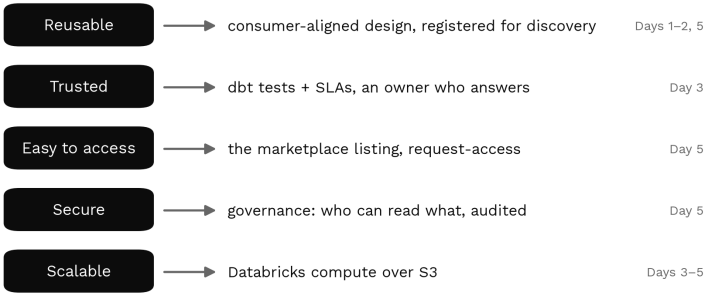
Each step reshapes the data away from the source's format and towards the consumer's question

- source = THEM: structured for the purposes of whoever produced it
- consumer = US: structured for the question our consumer needs answered
- the data creator sits in the middle and does the realignment

Morningstar's business is book-of-record data for stock-market events; our customers need it grouped by funds and portfolio managers. Morningstar doesn't have our customers, so its data isn't shaped for them. Realigning it *is* the data product. The terms are relative: source-aligned to them, consumer-aligned to us.

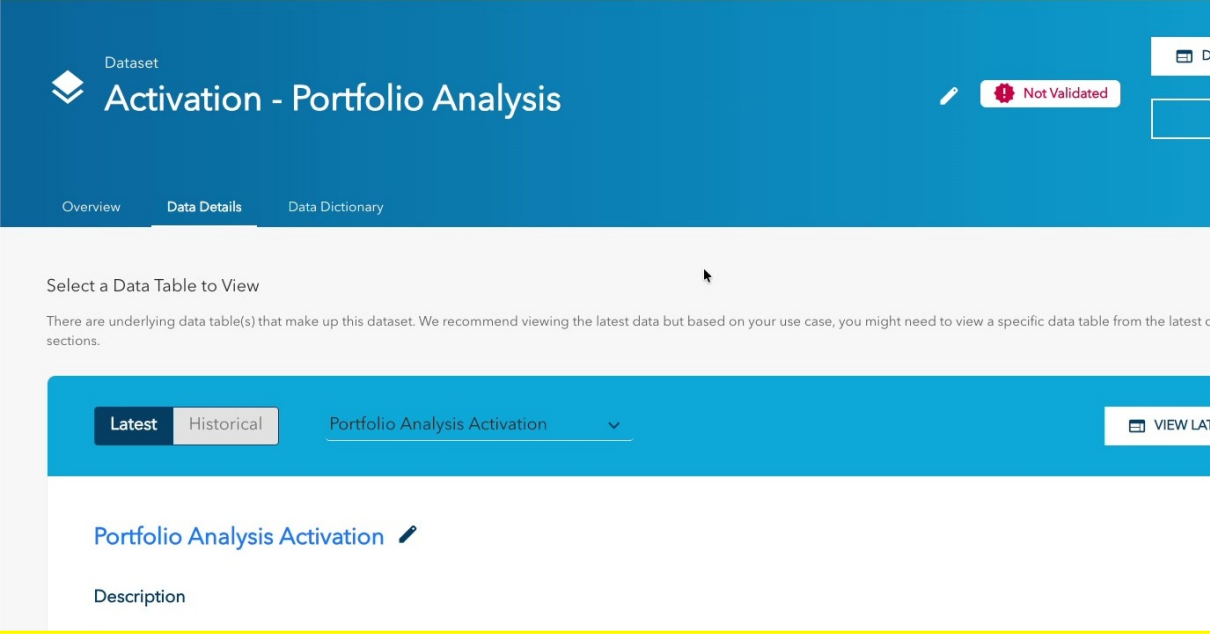
D1.5 What Do Data Products Promise?

A promise is an obligation between two parties.



Each promise becomes machinery you will build or configure this week

A data product is not a data set, and not a property of a data set: it consists principally in the promises. When we ship one, we release something that will have a certain quality, arrive at a certain time, and have an owner who responds to emails. Otherwise it's a dump of data, and no one cares about that.



D1.6 Promises, Made Visible
Match them up: the glossary tab is “trusted”, request-access is “secure”, the latest/historical split and refresh meta-data are quality and freshness, and the owner is who the promises are from. Day 5 returns to this page from the producer’s side.

D1.7 Worked Example: A Vendor ESG Data Product

MSCI, vendor providing ESG datasets: CO_2 emissions, people properties, ...

- $CO_2 \rightarrow$ exposure risk: e.g. exposure to future carbon taxes
- SLA: data *will* be available at a certain time, met 95% of the time
- Data owner: yes; governance, who has access, quality validation

Note the 95%: if the promise itself says “95% of the time”, then hitting 95% is *keeping* the promise, not breaking it. The promise sets the expectation.

D1.8 Two Questions Before You Build

- Should we create a data product?
- What data product should we create?

A canvas, the industry term for a grid of design questions and prompts:

- Design canvas: *should* we?
- Product canvas: *what*, and *how*?

We bounce between the two questions, because you need to know *what* the product is before deciding *whether* to build it. There's a level of circularity, and the canvases are how you work through it on paper before anyone writes code.

D1.9 How Do We Find Out What to Create?

Ask the consumer for a *problem statement*:

Channel	*Method*
Email	survey
Call	interview
Observe	what / when / why?
Research	documentation

Each method fails differently. Surveys are cheap but framed in the technologist's terms, not the consumer's. Interviews are dynamic but low-bandwidth and easy to overfit. Observation reveals what asking cannot ("revealed preferences"), but people do what the *environment* makes easy, which can disguise what they want.

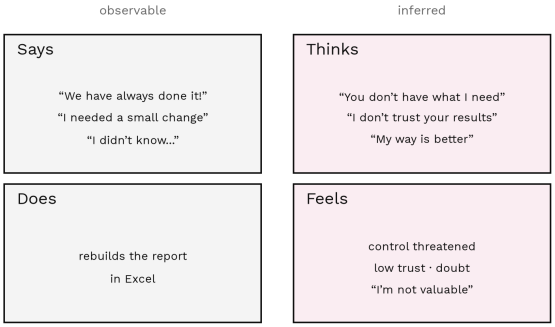
D1.10 Scenario: Adoption is Low

We have made a data product, but adoption is low...
We investigate... dum... THUNDER...

We find the consumer using Excel to reproduce a report we already make.

This scenario is real and common. Resist the first explanation (“they don’t get it”). The empathy map on the next slide is how we find the actual one.

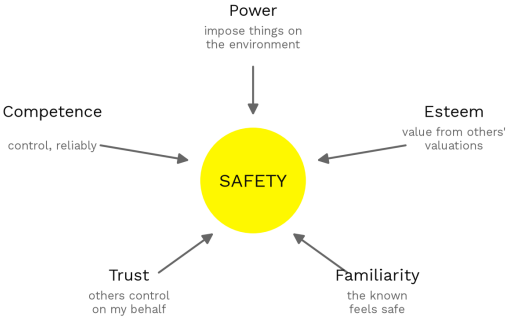
D1.11 The Empathy Map



Says and Does you can watch; Thinks and Feels you must infer, and the feelings drive the behaviour

All the design methods are hard for the same reason: there is a gap between the truth (what people really need, want and do) and what they say and visibly do. The top row is observable; the bottom row is inferred; and notice that the feelings mostly bottom out in *safety*.

D1.12 Aside: How Do We Manage How People Feel?



Every other motivation is a strategy for safety

Adoption asks people to trade their own control (“my spreadsheet”) for trust in yours, and delegation is inherently threatening. Propose solutions without understanding how people feel and they lie more, shut down, or become hostile; we have an obligation not to create situations where people won’t use what we build.

D1.13 Challenge 1.1: Data Product Role-Play

In threes, 30 minutes: consumer, producer, data creator. The creator interviews both:

1. What decisions depend on the data?
2. What happens when it is late, wrong, or missing?
3. What workarounds exist today?
4. What would ideal data delivery look like?

Output: interview findings + an empathy map for at least one pain point.

Make the person most familiar with the data the *consumer*, not the creator. An engaging conversation can displace its purpose, so keep the empathy map in view and complete it before the interview ends. This gives you evidence about decisions, workarounds and concerns before defining a product the consumer may not adopt.

Day 1 · Afternoon: Catalyst and Generative AI



D1.14 Lunchtime Task: Catalyst Setup

By the end of lunch, everyone should *have* Catalyst. From the onboarding material, install the Catalyst marketplace plugin for Copilot, then test:

```
run /data-contract
```

You should see a Catalyst *skill* called `data-contract` start asking questions.

If it's installed and this doesn't work, come and find me from 1pm. A GitHub Enterprise sign-in issue can recur after an earlier successful run and needs platform support. If the skill starts and asks questions, it's working.

D1.15 What is AI?

Artificial Intelligence: an attempt to solve problems with machines that *presently require human intelligence*.

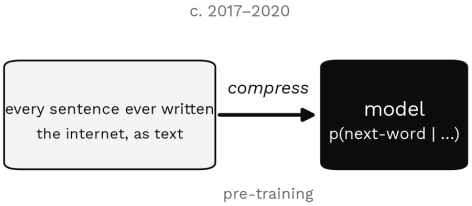
Two kinds matter today:

- discriminative = *labelling* existing content
- generative = *generating* new content

LLM, Large Language Model: an LLM is a model of language.

Spam filters and fraud flags are discriminative; Catalyst is generative. And language itself is a very low-information system (the symbols T-R-E-E have nothing to do with a tree, no bark, no green), so it is remarkable that analysing text alone learns anything at all.

D1.16 How Does an LLM Model Language?



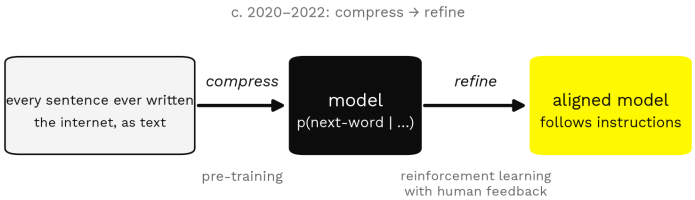
p(next word | prev words, TheInternetAllText)

The compression is the model: predicting your next word is decompressing the corpus

- trained on as close to *every sentence ever written* as exists: the internet
- the training compresses that text; the compression *is* the probability model

Ideally the vendors would train on every sentence ever written: predicting your next word would then just be looking up this conversation in the book of everything ever said. The internet is the substitute: a bad choice, but the biggest available.

D1.17 How Does a Modern LLM Work?



p(next word | prev words, TheInternetAllText, Instructions, Preferences)

Compression gives the model knowledge; refinement makes it follow instructions and preferences

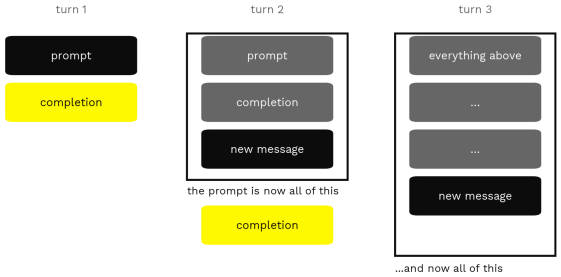
1. Pre-training: compress the corpus (c. 2017–2020, “training”)
2. Instruction fine-tuning: thousands of annotators write instruction/response pairs
3. RLHF: experts rank pairs of responses; the model learns the *preferences*

Why stage 3? There are near-infinite ways to follow an instruction correctly but badly: if told to bake a cake, burning the cake is still baking a cake. Stages 2 and 3 are now called *post-training*, and stage 1 *pre-training*, which tells you where the focus has moved.

D1.18 Talking to an LLM

```
response = llm(conversation)
```

- non-deterministic: the same prompt can give different completions
- known failure modes: hallucination, inconsistency

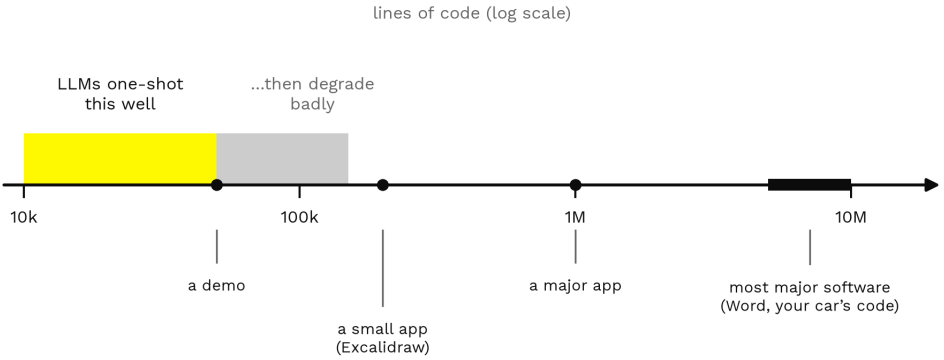


The prompt is the entire conversation so far: nothing else is remembered

As a conversation grows, the prompt *becomes the entire conversation*: the input is always everything so far. You can't solve non-determinism, but you can tame it with reference prompts, crafted and tested until the output is reliably roughly right. A Catalyst skill is exactly that, curated.

D1.19 Why AI Is Unusually Good at Code

Code is the one domain where preferences can be automated: the computer itself judges whether the programme runs and the output is right.



Maths, recipes and poetry need a human to say "this one's better"; code can be reinforced over billions of examples. That's why Catalyst is reliable for data-product code. The line counts are the caveat: engineers use LLMs to generate *parts* of systems and stay involved in producing the whole.

D1.20 Challenge 1.2: Generate a Data Contract with Catalyst

Pairs or triples, 45 minutes:

1. Describe your product, source, and consumer need in the prompt (not a bare `run /data-contract`); try a short and a long version on separate laptops
2. Revise the contract in the conversation, section by section
3. Compare, converge on one, and finish with *"save my data contract to a file"*

Keep the file: Day 2 starts from it.

The point of generating separately is to see how prompt length changes the output. You need to correct a generated design because Catalyst cannot verify that its requirements, schema, or promises match the consumer's need.

D1.21 How the Tooling Works: the AI Harness, Models, Effort

An LLM only generates text. The harness (Copilot) wraps it: observes the response, interprets tool calls, runs them, feeds results back.

- Agent vs Plan mode: generate-and-run vs say-what-you-would-do
- Approvals: default · autopilot · bypass (dangerous)
- Effort levels: hidden reasoning tokens; higher is not always better
- Vendors: choose empirically; run *your* job and compare

Two similarly named things: the lowercase word on this slide means Copilot's wrapper around the model; capital-H Harness is the CI/CD product we meet on Days 3 to 5. At Capital Group the sensible approval modes are default or autopilot. Rule of thumb on vendors: OpenAI leans literalist (instruction-tuned), Anthropic leans on preference quality; test the current models rather than trusting the folk theory.

Day 1 · End of Day: Wrap-Up

DECODED

D1.22 Capital Group, Investment Data, and Atlas

- Capital Group: an active investment manager; funds, ETFs, managed and retirement accounts
- This course makes that concrete with portfolios, funds, holdings, trades, exposures (e.g. US exposure 0.60, from a Morningstar-style row)
- Atlas is a separate *fictional* wealth-management platform used for the synthetic capstone (`instruments.csv`, `trades.csv`)

A source-aligned row keeps the producer's shape; we conform identifiers and meanings in a domain-aligned layer, then publish only what the consumer needs. Appendix E carries the plain-English glossary and the worked Morningstar join.

D1.23 Day 1 Recap

- A data product = data set + promises (SLAs, ownership, governance)
- Three linked views: promises · assets · alignment
- Requirements come from interviews, empathy maps, observation
- An LLM = a compressed, refined model of language; non-deterministic
- Code is where AI is strongest, because reinforcement is automatable

Tomorrow: the design canvas, effective AI, the contract, the paved path.

The closing warning: Catalyst can quickly produce something that *looks like* a data product, and that convincing first result displaces the skepticism you need to review it. Keep the casino mindset in view before repeated generation makes you less critical.

Day 2: The Design Canvas, Effective AI, the Data Contract, Adoption, and the Paved Path

DECODED

D2.1 Opening Poll, and the Plan

Raise your hand if you have:

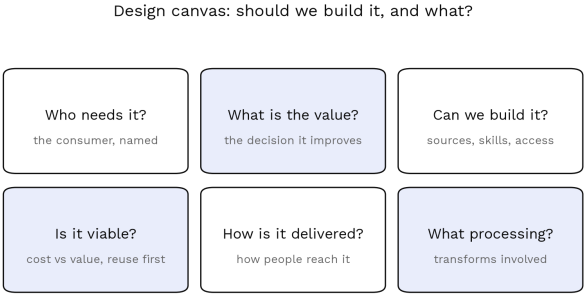
- written or reviewed source-to-target transformation rules
- worked with a service level indicator (SLI)
- worked with a service level agreement (SLA)

Then change table groups: cross-team conversation is part of the first two days.

A typical result has the most hands for SLAs and the fewest for SLIs. The key takeaway restated from yesterday's close: assuming every organisation has the same baseline models, the differentiator between AI systems is the *skills* built on top of them.

D2.2 Challenge 2.1: The Design Canvas

In groups, 30 to 40 minutes, complete the canvas for your chosen product:



Six questions that decide whether to build, before anyone writes code

Focus on the areas we discuss least: delivery, viability, and processing. And remember the canvas is a decision tool, not paperwork: a canvas that concludes “reuse an existing product” is a *successful* canvas. Report back only what was difficult or surprising.

Day 2 · Morning: Effective AI, Data Contracts, and Prioritisation

DECODED

D2.3 Effective AI: What Hallucination Really Is

In an LLM there is no second mechanism: the model is always doing the same thing, computing a completion from internet data, instruction data, and preference data.

- no mode switch between “telling the truth” and “making things up”
- the mechanism that generates correct tokens *is* the one that generates incorrect ones
- so nothing inside an LLM can make it “not hallucinate”

In that sense LLMs are *always* hallucinating; sometimes you like the output and sometimes you don’t. Whenever we work with these systems we should simply expect this behaviour, which is why review is a design requirement rather than a nicety.

D2.4 Context-Window Bias

The model places a learned importance distribution over the prompt, and it over-weights the start and the end; attention to the middle fades.

- as the prompt grows, your important information ends up in the middle
- instruction words (“write”, “draw”) get enormous early weight
- the entire conversation is always the input; a new conversation resets it

Like handing someone a book and asking what it’s about: they read the first page and the last page and skim the middle. If a chatbot seems to remember across threads, that’s the harness using hidden prompts, memories, or retrieval, not the model.

D2.5 When to Be Skeptical

1. Long conversations: expect contradictions with earlier content
2. Novelty: your paved path applied to your Morningstar dataset is unlikely to be documented anywhere the model has seen

And a caveat with teeth: AI models can report work inaccurately. A model may claim everything works while every function returns its arguments.

A productive workflow: let a long generation complete, then treat the result as a *prototype* to test for errors before restarting. Avoid the casino mindset of repeating the same generation hoping the next result improves by chance. And never assess what a model did from the conversation alone: inspect the files and run the tests.

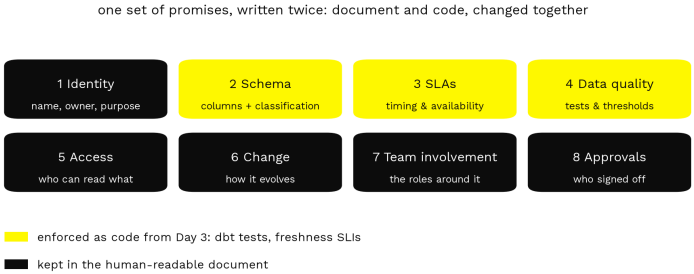
D2.6 Tips for Using AI Models Well

1. Use the model to critique itself (or cross models): drafting and critiquing activate different weights
2. Plan, and plan to plan: never “go and do X”; ask for the plan first
3. Understand steering: an LLM *cannot ignore* your words; add dangerous specifics late
4. State the future use of an output (“later I’ll use this for Y; don’t do Y now”)
5. Borrow the model’s metacognition: “what might I not know here?”
6. Know when to start fresh: a new conversation that subtracts

Steering is why skills work: a skill is a handcrafted steering system, key words placed to pull the model toward the right completion. A human can challenge or ignore a misplaced detail; the model averages it in, so sometimes removing words helps as much as adding them.

D2.7 The Data Contract: Two Meanings, Eight Sections

“Data contract” means two things: the human-readable document below, and automatic code enforcement of the same promises (Days 3 to 5).



The yellow sections are the ones Day 3 turns into dbt tests and freshness SLIs; the rest stay in the document

Someone designing a product who says “we need a data contract” means the document; an engineer who says “I’ve written a data contract” means code. Both halves express the same promises and must change together. A filled eight-section exemplar is in Appendix A.

D2.8 Challenge 2.2: Interview, Empathy Map, and Catalyst Contract

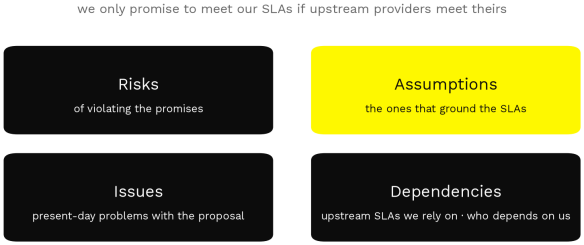
Groups of three, 25 to 30 minutes:

1. One consumer (their *real* role, a need the interviewers don't understand), two data creators
2. Generate an example contract with Catalyst
3. Interview, take notes, map says/thinks/does/feels for one pain point
4. Give the evidence to Catalyst, regenerate, then review section by section

The framing for the whole week: we are teaching what you need to know to double-check what Catalyst generates, not teaching Catalyst itself. In one run the generated contract's *schema* was the least accurate part; what would you add to the prompt to prevent that?

D2.9 Prioritising Promises: RAID

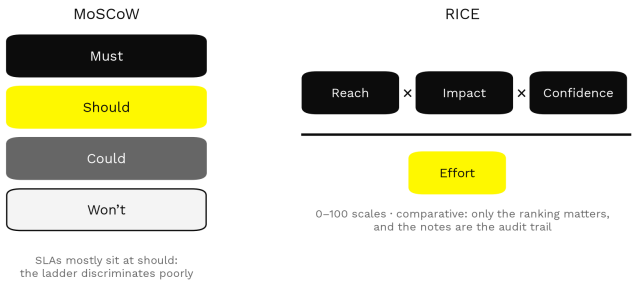
Risks · Assumptions · Issues · Dependencies: assessing the degree to which you can keep your promises.



Assumptions is the quadrant that matters most: it holds whatever your SLAs silently depend on

We only promise to meet our SLAs if upstream providers meet theirs, which is why the assumptions row matters most. Teams run RAID before starting a PI, reassess at leadership sync-ups, and carry residual risks forward.

D2.10 Prioritising Promises: MoSCoW and RICE



MoSCoW ranks categories and stalls at should; RICE ranks the individual trade-off when one promise has to give

The formulas are *comparative*: all that matters is that they rank the promises correctly, and that you keep the notes.

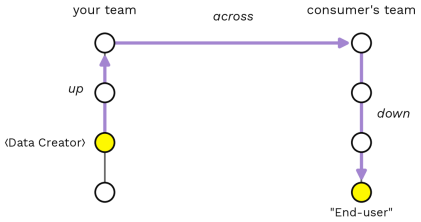
The value is accountability and transparency rather than better cognition: in a large financial organisation you must account for decisions through an audit trail, and "why did you decide that?" is harder to answer well on the spot than the decision was to make. Worked case: being ten minutes late on the timing SLA to deliver the explanation, defensibly.

Day 2 · Midday: Adoption, Stakeholders, and Interviewing Catalyst

DECODED

D2.11 Adoption is a Leadership Problem

- Management: given a policy, organise the business to execute it
- Leadership: *set* the mission, stay close to where people are, bridge the gap
- Innovation: out ahead of everyone, principled, stubborn, and frightening

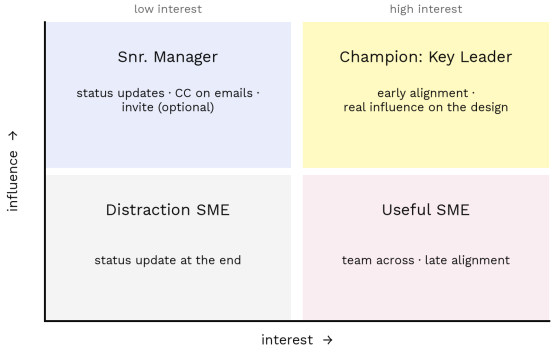


influence travels up, across, and down other teams' chains

You lead change you cannot mandate: influence goes up your chain, across, and down theirs

Innovation and management don't solve adoption problems: innovators demonstrate practices miles from where people are, and management just tells people what to do. Often the leader's real job is to pick an innovator they trust and bring people along that path. These five days are themselves an adoption exercise for the paved path.

D2.12 Mapping Your Stakeholders



The quadrant sets the strategy: early alignment for champions, bracketed updates for everyone else

We manage outward.

Watch the high-interest, low-influence quadrant: it's pleasant to talk to people enthusiastic about your work, so that's where the meetings go, and we mistake it for progress. Bracket them with early and late alignment instead. And you are on this chart too, somewhere in the middle, managing outward.

D2.13 Challenge 2.3: Getting Buy-In

Groups: choose an existing product, propose a version 2, then work out:

1. Who could block it; who is affected, positively or negatively, and why
2. Who belongs in design meetings, who is excluded, who is kept informed
3. Assume it still fails: which trust, power, control, or alignment condition did your analysis miss?

Behind most resistance sits esteem, power, and control. Nobody ever *says* “I don’t want to change”; resistance presents as an argument with merit, usually “it’s unreliable”, and rebutting it can make behaviour worse. The way in is the empathy map: address the motivation, not the stated argument.

D2.14 Challenge 2.4: Interviewing Catalyst

Individually, 15 minutes. Ask Catalyst:

1. What is the data product lifecycle at CG? Phases and requirements?
2. What is a data product? What skills do you provide?
3. Can you search Databricks, or existing data products?
4. What parts of the lifecycle can you *not* help with?
5. What technologies does the paved path use, and what does each do?

Checkpoint: Catalyst should report nine phases.

Catalyst's skills encode CG guidance, so when you don't know the policy, ask it, then confirm with the policy owner. And remember this morning's caveat: the conversation is not a reliable record of what the model actually did.

Day 2 · Afternoon: The Paved Path, the Dev Container, and Docker

DECODED

D2.15 The Paved Path

The standard Capital tooling for data products:

Tool	*Plain-English job*
git / GitHub	code, history, and the pull-request record
Docker + dev containers	a repeatable environment and its dependencies
dbt	SQL + templates + tests: source data → consumer-aligned models
Airflow (Astronomer)	schedules, orders, and monitors pipeline tasks
Databricks (UC, Marketplace)	runs transformations over S3; govern and find products
Harness	releases code and drives the deployment pipeline
Catalyst	guides lifecycle work; routes requests to skills

“Paved path” means the supported, pre-approved route: go off-road and you own every problem yourself. PySpark was on the path and the direction now is to move everyone off it towards dbt. Harness is the one tool with no hands-on challenges, because it sits behind the credit card: it provisions real infrastructure.

D2.16 Git and GitHub

Source control is versioning software: version code, code history, sharing.

GitHub is not Git: it's a platform built on Git, the *"single source of truth"*, the book of record for code.

Pull Request (PR): a request to take one version of the code and integrate it into, or replace, another.

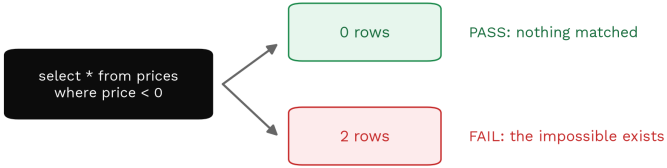
At CG the PR is part of the permission structure: the pull request is often where acceptance for a release actually happens, which makes GitHub a key part of the permissions process. On Day 5 you'll see the same merge event trigger deployment.

D2.17 dbt in One Slide

dbt (data build tool): data transforms as SQL + templates + tests.

This is the tool that performs the source-to-consumer alignment.

Q. How do you test SQL?



the query asks for rows that should never exist

A test is a query for rows that must never exist: zero rows is a pass

A dbt test is just another query, written so that it returns rows only when something is wrong. Zero rows means the test passes. A core part of dbt testing is writing queries that should never produce data.

D2.18 Airflow and Databricks in One Slide

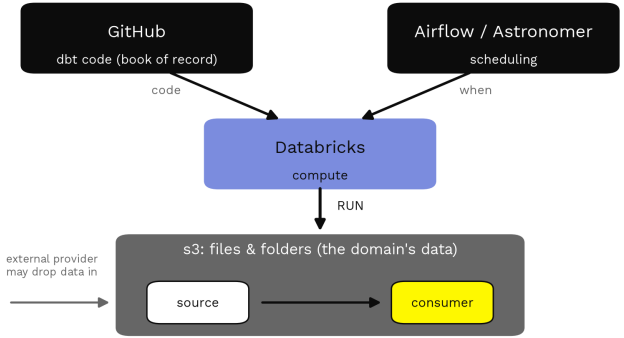
Airflow: an orchestrator. Think of a conductor: while the orchestra plays, the conductor is *timing, cueing, and coordinating*. Airflow times, cues, and coordinates tasks.

Databricks runs the rules:

- dbt = *rules* to transform data
- Databricks = *runs those rules* on data inside files & folders (AWS S3)

For Linux people, Airflow is cron; for everyone else, it's Windows Task Scheduler. And the strong argument for Databricks today is vendor integration: Morningstar and Bloomberg will simply drop files into the folders. With a traditional database, vendors would need credentials and insert access to our systems.

D2.19 How the Paved Path Fits Together



Code from GitHub, timing from Airflow, compute on Databricks: the product is the realigned folder

At the end of the day, it's a folder somewhere that something is copying files into and out of, changing their alignment along the way. The tooling is what makes that reliable, permissioned, and repeatable. Every remaining day zooms into one box of this figure.

D2.20 Dev Container Setup

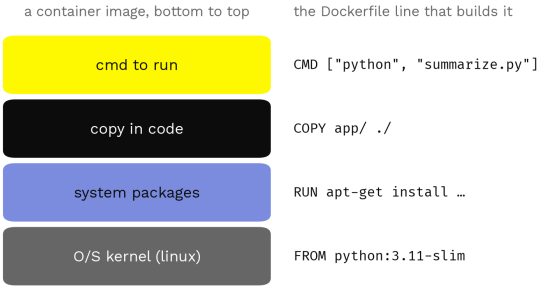
From the onboarding link: clone the EDO data-products repository, open in VS Code, accept “Reopen in container”.

Inside: a challenges folder: `01` Docker · `02` DBT · `03` Airflow.

Do this today so problems reach platform support *today* rather than blocking tomorrow. Known issues: some Windows laptops need Docker’s RAM allocation adjusted, and the repo contains two Dockerfiles; the root one is the one to build from.

D2.21 Docker in Depth: Two Roles, Four Layers

Docker plays two roles on the paved path: development (bundle dependencies so code runs anywhere) and release (to make software live is to deploy the container).



Each Dockerfile line adds a layer: the image is a recipe, executed bottom to top

We release the bundle itself, so the development environment mirrors the production environment. From the outside we treat a container as if it were just the command; if the command were `2 + 3`, running a container would be an absurdly expensive way to get 5, so the commands are worth the overhead: a dbt run, an Airflow server.

D2.22 Dev Containers: What's in Ours

Dev container = a Docker container + VS Code running *"within"* the container.

- mise (*en place*): cross-language dependency manager
- astronomer: airflow + container (*"containers in containers"*)
- databricks cli: used by dbt, and by Catalyst when it runs queries
- docker, dbt

Does a data creator need to build containers? Mostly no. This container, prepared by the EDO team, is the one you really deal with: you create dbt code inside it and submit to Git. Deploying containers to live systems is outside the data-creator role, but awareness is the point.

D2.23 Day 2 Recap

- The design canvas decides desirable, feasible, viable, deliverable
- Effective AI: no second mechanism; the middle of the prompt fades; verify the files
- The contract: eight sections, human half and enforced half
- RAID, RICE, MoSCoW make trade-offs visible; they don’t replace judgment
- Adoption fails on leadership and alignment, not technology
- The paved path toured; the dev container running

Tomorrow the rhythm changes: roughly a 20-minute introduction to a tool, then an hour or more of self-paced challenges, cycling through Docker, DBT, and on Day 4, Airflow. Platform support is on hand for setup issues.

Day 3: Docker, Data Systems, and DBT

DECODED

D3.1 How Days 3–4 Run, and the Poll

The rhythm from here: 20–30 minute overview → 45–60 minute self-paced challenge block → regroup.

Hands up if you have used, in a non-trivial way: Git · Docker · DBT (and SQL: a two-table join and a `GROUP BY` unaided) · Airflow · Databricks · Harness.

If you already know a tool, move quickly through the set challenges or use the time for advanced work prompted through Catalyst. For every challenge README, right-click and Open Preview: some later files hold solutions inline, and the preview hides them while the raw view exposes them.

D3.2 The Line: Development vs Live

Above the line, the product is essentially done: dbt, Docker, Git, GitHub (the code, the container, the record). Below the line it runs:

- Databricks: the release target; where products are discovered and read
- Airflow: triggers the product's code to run
- Harness: the wrapper around all of it; a data creator's detail can stop here

What you will mostly be doing is writing dbt code.

The working loop is short: you receive a container, a dbt project and sometimes Airflow configuration; you modify the dbt code, create the STTM that records the rules, and push the change. Python's absence from the list is deliberate: a data creator can often avoid writing Python altogether.

D3.3 Challenge 3.1: Branch, Commit, and Read the Diff

Individual, about 30 minutes, in the courseware repo:

1. `git log --oneline -20`; read the messages
2. `git switch -c rename-exercise`
3. Change one rename in a staging SQL file
4. `git status`, stage, commit with a message that describes the rename
5. `git diff main`; read the removed and added lines
6. `git switch main`; confirm the commit exists only on your branch

The diff you just read is the same evidence a pull-request reviewer reads, and every command here returns during the DBT block, which makes this a rehearsal rather than a detour.

Day 3 · Docker: Containers, Dockerfiles, and the Challenge Block

DECODED

D3.4 Demo: Anatomy of a Dockerfile

challenges/01-docker-intro/Dockerfile

```
# Start from a small official Python base image (Python 3.11 on slim Debian).
FROM python:3.11-slim

# Everything below runs relative to /app inside the image.
WORKDIR /app

# Copy the app (summarize.py + holdings.csv) into the working directory.
COPY app/ ./

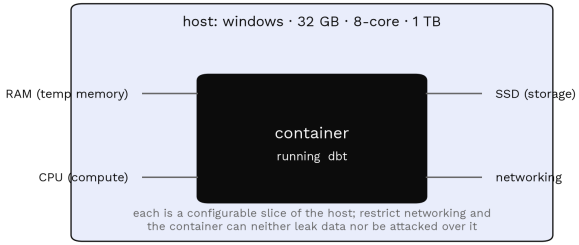
# Run the summary script when the container starts.
CMD ["python", "summarize.py"]
```

Four instructions are a complete container: FROM chooses the base image, WORKDIR and COPY bring the code in, CMD names the one command the container exists to run

Each line produces a layer, and layers are cached: change your code and only COPY onwards rebuilds, which is why builds after the first take seconds. The production Astro Dockerfile is the same idea with CG certificates and registries baked in.

D3.5 Configuring a Container's Resources

When I run a container, what can I configure?



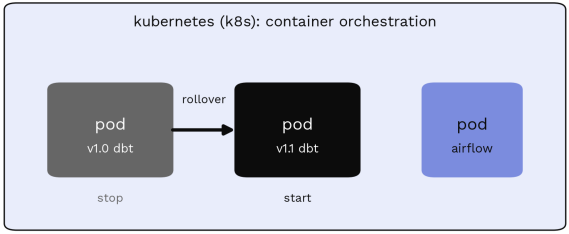
Each slice is a cap: a container that exceeds it fails alone instead of taking the host down

Q. Why is it useful to restrict networking?

In production this is the main event: a container capped at 2GB that starts demanding more has something wrong with *it*, and it fails alone instead of starving the host. Networking is the most important control: assume you *will* be hacked, and a compromised container that's forbidden from talking to anything has strictly limited blast radius.

D3.6 Aside: Kubernetes, Infrastructure as Configuration

k8s = container orchestration: starting, stopping, timing containers.



Amazon EKS = managed k8s · at CG: ECS & EKS today, Helix next

k8s starts and stops containers from a config file: a rollover is stop v1.0, start v1.1, no typing

At CG: ECS and EKS both allowed... many moving to EKS. Future: Helix.

Yesterday we started containers by typing `docker run` with our meat-based fingers. Kubernetes replaces that with configuration files defining many containers and their networking: a complicated many-step problem turned into a plain-text file, which can be versioned in Git, diffed, and automated. Mouse clicks can't be.

D3.7 Challenge 3.2: Build and Diagnose a Docker Container

From the `01` folder in the dev container: build your own image and container, then diagnose what the challenge breaks.

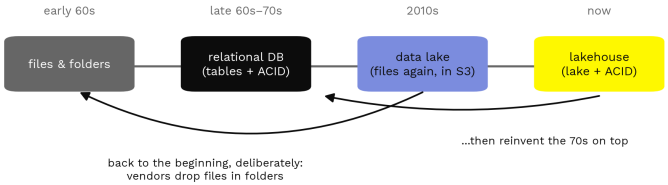
Finished early? Investigate multi-stage builds, container networking, and EKS.

This is one of the few times you'll be gifted time to improve your Docker knowledge, so use it. The recurring Windows issue is Docker's memory allocation; platform support is in the room.

Day 3 · Data Concepts: Systems, Pipelines, and Design Artefacts

DECODED

D3.8 Data Systems: Database, Warehouse, Lake, Lakehouse

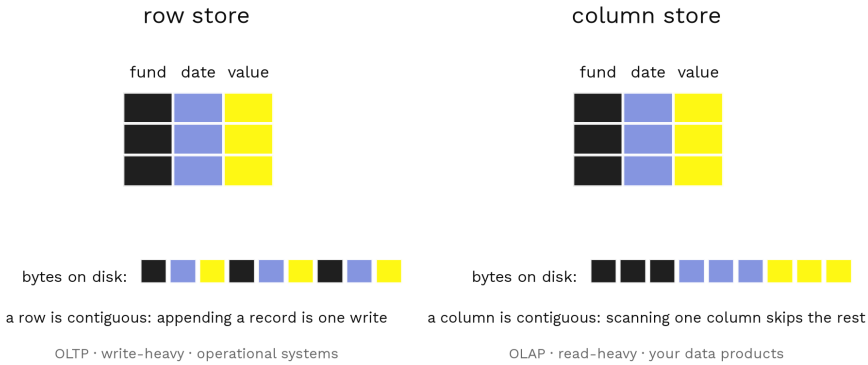


The loop: files, then databases for the guarantees, then files again for the vendors

- database: structured (= tabular) storage; a relation is a table
- warehouse: an analytical (OLAP) database that also tracks history
- data store / lake: files & folders; flexible formats (Parquet), low cost
- one trend across all of it: increasing flexibility of storage

“Structured always means tabular” in this discipline, confusingly, since an image is obviously structured too; that’s the history of the field. When you hear “data store”, you can almost always just think files and folders.

D3.9 OLTP vs OLAP: Rows vs Columns



The layout follows the workload: append rows to write fast, group columns to read fast

The original OLTP databases were retail databases, and the transactions were literal financial transactions; today the word means any write that must succeed or fail as a whole. Rule of thumb: write-heavy databases have few columns and many rows; analytical databases have many columns, parallelise well, and you very rarely see a GROUP BY in a transactional database.

D3.10 The Lakehouse, and ACID

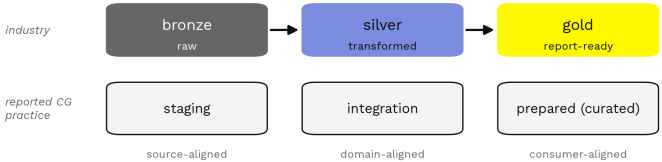
A lakehouse reimplements relational-database capabilities on top of a data lake: the storage stays files (Parquet); the *querying layer* adds SQL and ACID.

ACID together = linearization under failure: write 1, 2, 3 and, whatever fails in between, you read back 1, 2, 3 or you're told it failed.

Almost all databases had ACID until roughly 2005; the distributed decade relaxed the guarantees for speed, everyone rediscovered why they mattered, and lakehouses added them back. Try it yourself: `DESCRIBE HISTORY` on any Delta table shows every write as a version, and `VERSION AS OF` reads an earlier state; that versioning is how ACID lives on plain files.

D3.11 Pipelines, the Medallion System, and CG's Names

A data pipeline is *automated* movement of data with an operation at every stop: ingest → transform → join/curate → expose.



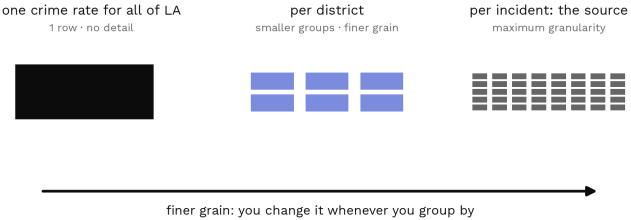
organise by alignment, describe by layer: the rows name the same tables

Two vocabularies, one pipeline: the industry's medallion names and CG's reported names label the same tables

Keep two axes in view: organise by *alignment* (whose shape is this built for?), describe by *layer* (how far has it been processed?). The rule: communicate and document in source/domain/consumer-aligned language; build with staging and intermediate, `stg_` and `int_`.

D3.12 Grain

The grain is the size of the group in a group-by: how coarsely you've averaged the data.



Every group-by moves you left: maximum granularity is generally at the source

Mostly a pipeline lowers granularity as data flows through the stages, which is why the contract states the published grain explicitly (one row per fund, day and instrument) and why changing it needs a consumer review.

D3.13 What is an STTM?

The data lineage from source to target: the business rules and transformational logic that take you from one to the other.

> *e.g. individual accounts: two to four sources JOIN → dedup → assign a unique internal ID*

How do you specify one? In English: source attributes and definitions (a *"glossary"*), target attributes and types, and the rules between them.

That unique-ID step is one of the genuinely hard areas. Think of police databases: names spelled differently across crime-scene reports, and no easy way to know whether two reports describe the same person (the "golden nominal" problem). Morningstar, Bloomberg and the rest each use their own IDs, and deciding whether they mean the same asset is the same problem.

D3.14 The Types of Transforms

Transform	*Shape*
direct / passthru	`c1 → c1`
join	`[c1, c2] → c1, c2`
derived	`c_new = op(c_old)`
aggregation	`c_new = avg(subset(c_old))`
filter	`c_new = subset(c_old, filter)`
union	rows of two source columns combined
lookup	`c_new = lookup_table[c_old]`, e.g. `GB → "United Kingdom"`
conditions	branching logic

What do these amount to? I've got a spreadsheet, I create a new sheet, and I want to populate it from the previous sheets. When a mapping row doesn't fit one of these shapes, the requirement isn't understood yet.

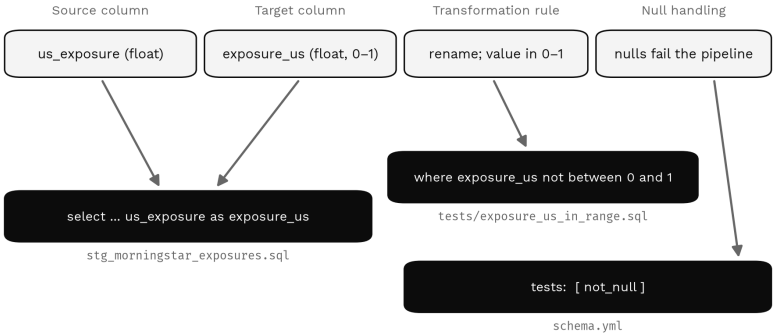
D3.15 The Gap Between English and Code

Where does an English STTM go wrong?

- *"pick a unique value"*: which policy? whose?
- vague requirements: *"firm name"*, legal or parent?
- null handling: what does null *mean* here? emptiness? zero-ness? defaults?
- complex conditions: does *or* mean either, or possibly both?
- scope of terms: "phone number", US only or worldwide?

Every bullet is a real bug class, and the fix is a question back to the requirement's owner, asked *before* building. On nulls: for every kind of data there is a different meaning of empty. English is ambiguous, so add words rather than relying on "if" and "or".

D3.16 Demo: One STTM Row, Implemented Live



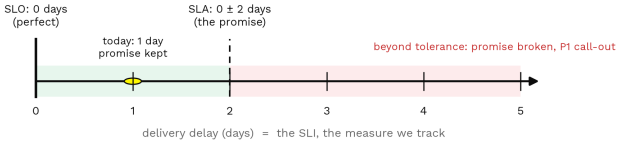
the mapping and the code carry the same information in two languages

Every cell of the mapping row lands in a file: the columns in the select, the rule as a zero-rows test, the null decision as not_null

Every cell of the row has a corresponding line: source column before the `as`, target column after it, the rule as test SQL, the null decision as the `not_null` entry. The mapping and the code carry the same information in two languages, and that correspondence is what makes the STTM auditable.

D3.17 SLAs, SLIs, SLOs, and RAID

- SLA, service level *agreement*: a promise between two parties
- SLI, service level *indicator*: the quantitative measure tracked
- SLO, service level *objective*: if the indicator were perfect, its value

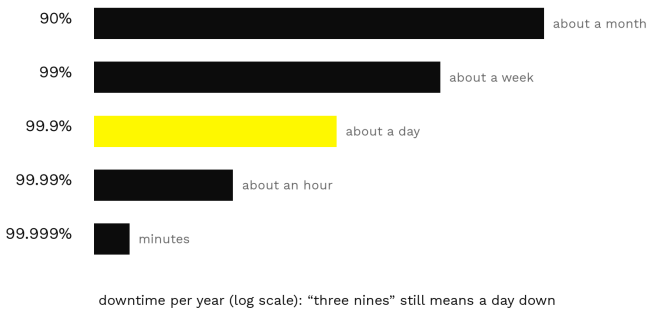


Measure it (SLI), target it (SLO), promise it with a tolerance (SLA)

Other examples: 7pm delivery *or a P1 call-out* · recovery time · a *“duty to explain”*.

The “duty to explain”: during the pandemic, oil futures went negative, which had never happened in recorded history. A trader seeing that number needs someone on the hook to say whether it’s real or a data-quality issue. For scored risks, RAID multiplies probability by impact (1 to 5 each); 16 or above needs immediate mitigation.

D3.18 What Does 99.9% Actually Mean?



Each nine buys a factor of ten: 99.9% still allows 8.76 hours down per year, about a working day

Saying “three nines” is saying “we can be down on the order of a day a year”. For calibration: Claude’s availability is about one nine, optimistically, and GitHub’s has fallen to roughly one nine too. Industry standard for serious systems is three to five.

D3.19 Challenges 3.3 & 3.4: Use Cases, and a PRD with Catalyst

Challenge 3.3 (groups): discover cross-team use cases for your table's products.

Challenge 3.4 (pairs, ~25 minutes): use Catalyst to generate a product requirements document for a named product, then iterate it in the same conversation: add a sample source-to-target map and a glossary.

The STTM is where the business rules, the actual IP, live, and that is exactly where the model is generic: it can fetch an accurate schema, but it doesn't know the goal of your product. Point it at a source of truth (it can search Confluence) and interrogate every rule it invents.

Day 3 · DBT: Setup, Guided Tour, and Challenge Block

DECODED

D3.20 Challenge 3.5: Predict Ten SQL Results

Ten minutes, individual, ungraded: predict the row and column shape of each snippet before revealing the answer:

1. `select id as customer_id, name as customer_name from customers` (then add a cast)
2. a `join`, then change it to a `left join`
3. a filtered `group by`, then add a second grouping column
4. the same query as a CTE, then stack a second CTE
5. `where amount < 0`, then apply it where negatives are valid

"DBT is basically SQL" only helps if the SQL is there to lean on; the tour assumes joins, GROUP BY and CTEs cold. Reading row-preservation and grain changes is necessary before accepting generated transformations or tests.

D3.21 How Does DBT Work?

dbt defines data transformations in mostly-SQL syntax, with extensions, and provides a testing approach.

- simplifies SQL down to just SELECT: each file's query defines a table named after the file
- the `{{ … }}` braces are the extensions, and they are always replaced by SQL

Two ways of connecting models: `ref` ("this depends on that model"; dbt builds the dependency tree and runs everything upstream) and `source` ("this depends on data dbt does not build"). Refs are what dbt chains together; sources are what must already exist.

D3.22 Key DBT CLI Commands

Command	*Purpose*
`dbt deps`	install project packages
`dbt debug`	check the profile and Databricks connection
`dbt parse` / `dbt compile`	validate syntax / compile models to SQL
`dbt seed`	load CSV seed files into the warehouse
`dbt run` (`--select <model>`)	build all (or one) selected models
`dbt test` (`--select <model>`)	run schema and data tests
`dbt build`	run and test together, in DAG order
`dbt docs generate` / `serve`	generate and open the documentation site

On `seed`: it lifts a CSV into a table, which is a development tool. Why would Morningstar's data be on your laptop? The legitimate production case is a small lookup table you created as part of the project.

D3.23 Creating DBT Models

Two models, staged:

- `stg_holdings.sql`, staging: clean and rename the source
- `fct_portfolio_returns.sql`, fact: the consumer-aligned answer

The next two slides walk one of each, from the challenge-02 solution.

CG policy: every model lays out its stages as named CTEs, sources at the top, joins in the middle, final transformation at the end, even when there's little to join. Self-documentation. When *communicating*, use the source/domain/consumer-aligned language; when *building*, it's `stg_` and `int_`.

D3.23a Demo: A Staging Model

```sql
-- stg_holdings: cast + clean only, no joins.
-- Grain matches the source: one row per (fund, position_date, isin).
with src as (
    select * from {{ source('custodian', 'holdings_raw') }}
),
renamed as (
    select
        trim(upper(fund_cd))                        as fund_id,
        cast(pos_dt as date)                        as position_date,
        coalesce(isin, 'CASH_' || upper(ccy_cd))    as instrument_id,
        cast(qty as decimal(18, 4))                 as quantity,
        -- … (3 more renames elided) …
        _extracted_at                               as _src_loaded_at
    from src
    where fund_cd is not null
      and pos_dt  is not null
)
select * from renamed
```

A source CTE at the top, renames in the middle, a bare select at the end; the file reads like the STTM that specified it, which is the point.

D3.23b Demo: The Fact Model

```sql
-- fct_portfolio_returns: the published daily-grain fact.
-- One row per (fund_id, position_date, instrument_id). `fct_` is the
-- fact-table prefix; consumers are the RM dashboard, compliance, and
-- the client app API.
with holdings as (
    select * from {{ ref('int_holdings_enriched') }}
),
funds as (
    select * from {{ ref('int_fund_attributes') }}
),
-- … (fund_mv and with_units CTEs elided) …
select
    h.fund_id,
    h.position_date,
    h.instrument_id,
    -- … (more columns elided) …
    u.total_mv_base / u.units_outstanding as calc_nav_per_unit
from holdings h
left join with_units u
    on u.fund_id = h.fund_id
```

Every ref() is an edge in the dependency graph dbt builds for you: run this model and dbt runs everything upstream of it first.

D3.24 Configuring DBT

Where dbt's behaviour lives:

- `dbt_project.yml`, the project: paths, materialisations
- `profiles.yml`, the connection: Databricks host, schema, credentials

Two files, two concerns: what the project *is*, and where it *runs*.

In this project, models in staging materialise as views and models in marts as tables, set per folder in `dbt_project.yml` and overridable per model. `dbt debug` checks the connection without touching the project.

D3.24a Demo: dbt_project.yml

```yaml
name: 'portfolio_analytics_challenge'
profile: 'portfolio_analytics_challenge'
# Layers are separated by +tags, not +schema: the training catalog gives
# you exactly one pre-provisioned schema, so every layer materialises
# into that one schema and is selected by tag (`--select tag:staging`).
models:
  portfolio_analytics_challenge:
    staging:
      +materialized: view
      +tags: ["staging"]
    intermediate:
      +materialized: view
      +tags: ["intermediate"]
    marts:
      +materialized: table
      +tags: ["marts"]
      # Z-ordering post-hook: after each mart build, co-locate rows by
      # the columns most queries filter on, so reads prune faster.
      +post-hook: "OPTIMIZE {{ this }} ZORDER BY (fund_id, position_date)"
```

Layers are folders with policies attached, so the models stay pure SELECT: staging materialises as views, marts as tables with an optimise hook.

D3.24b Demo: profiles.yml

profiles.yml

```yaml
portfolio_analytics_challenge:
  target: dev
  outputs:
    local:
      type: databricks
      method: http
      catalog: "{{ var('app_catalog', 'training_dev') }}"
      schema: "{{ env_var('DATABRICKS_SCHEMA', var('target_schema', env_var('USER', 'dbt_user'))) }}"
      http_path: "{{ env_var('DBT_HTTP_PATH', var('http_path', '/sql/1.0/warehouses/a0e57263293aee36')) }}"
      host: "{{ env_var('DBT_HOST', env_var('DATABRICKS_HOST', 'https://cg-db-aa00003661-dev-us-east-1.cloud.databricks.com')) }}"
      token: "{{ env_var('DBT_ACCESS_TOKEN', env_var('DATABRICKS_TOKEN', 'MISSING_run__mise_run_setup_databricks')) }}"
      connection_parameters:
        _disable_pandas: true
# … (20 lines elided) …
```

Every credential is an env_var lookup with a fallback: nothing secret is written in the file, and the same profile shape serves every target

`dbt debug` checks exactly this handshake. When it fails, the fix is almost always an environment variable (host, token, http_path), not a code change.

D3.25 Writing DBT Tests

Two families:

1. YAML (schema) tests: `not_null` · `unique` · `relationships` · `accepted_values`
2. Query tests: a select whose expected result is zero rows

Why define tests in YAML at all? Because some things are hard to test in a select query: that columns are named correctly, that nulls are handled. Add tests as you write each model; they are your STTM edge conditions, made executable.

D3.25a Demo: Tests Beside the Models

```yaml
# models/staging/schema.yml
models:
  - name: stg_funds
    description: "Cleaned fund master, one row per fund."
    columns:
      - name: fund_id
        description: "Primary key — fund identifier."
        tests:
          - not_null
          - unique
  - name: stg_holdings
    description: "Cleaned holdings, one row per (fund, day, instrument)."
    columns:
      - name: fund_id
        tests:
          - not_null
      - name: instrument_id
        tests:
          - not_null
# … (position_date and later models elided) …
```

The tests are declarations next to the model they guard, and each compiles to the same kind of query as everything else in dbt: select the violations, pass on zero rows. The descriptions feed the docs site and the glossary.

D3.25b Demo: Tests on the Marts

models/marts/schema.yml

```yaml
# … (10 lines elided) …
models:
  - name: fct_portfolio_returns
    description: >
      Daily-grain fact: per (fund, date, instrument) position with base-ccy
      market value and a calculated NAV-per-unit. One row per
      (fund_id, position_date, instrument_id).
    columns:
      - name: fund_id
        description: "Fund identifier."
        tests:
          - not_null
          - relationships:
              arguments:
                to: ref('int_fund_attributes')
                field: fund_id
      - name: position_date
        description: "Calendar date the position is valued on."
        tests:
          - not_null
      - name: instrument_id
        description: "ISIN, or synthetic CASH_<ccy> for a cash holding."
        tests:
          - not_null
      - name: mv_base_ccy
        description: "Market value converted to the GBP base currency."
        tests:
          - not_null
# … (1 lines elided) …
  - name: agg_fund_exposure
    description: >
      Per (fund, date, strategy) exposure aggregate. Pre-computed measures by
      reporting dimensions, used by compliance for concentration alerts.
    columns:
      - name: fund_id
        tests:
          - not_null
      - name: position_date
        tests:
          - not_null
      - name: strategy_code
        description: "Strategy classification carried from the fund master."
        tests:
          - accepted_values:
              arguments:
                values: ['EQ-LC', 'EQ-EM', 'EQ-GC', 'FI-IG']
# … (8 lines elided) …
```

The curated layer carries the stronger promises: `relationships` proves every fund exists in the fund master; `accepted_values` pins strategy codes to the known set

Beyond not_null and unique: `relationships` is referential integrity as a test, and `accepted_values` freezes the vocabulary a column may use. Both compile to the same select-the-violations query; both live beside the mart they guard.

D3.26 Challenge 3.6: Build the DBT Pipeline

The `02-dbt-pipeline` route: eight guided sections (S1–S8), then ten challenges (C1–C10). About an hour for the guided route; continue as time permits.

Done when: the project passes `dbt debug`, seed, run, and test verification, and includes the capstone mart.

This is the transformation code a data creator is most likely to own: sources become documented, tested warehouse products whose lineage can be reviewed. Skip the pipeline-overview diagram on first reading; it's easier after you've built the models. Stretch topics: incremental models, snapshots, model contracts, unit tests, macros.

D3.27 Day 3 Recap

- The line: dbt, Docker, Git above; Databricks, Airflow, Harness below
- dbt keeps transformation logic in SQL; each model is an implied create
- `source` and `ref` build the dependency graph
- Tests turn contract promises into queries that pass on zero rows
- **`dbt build`** interleaves run and test in DAG order, so nothing downstream is built on data that failed its checks

`dbt build` is the command that makes the contract binding during the build itself, and in production it is usually the command the scheduler runs. `--select` targets a layer, folder, model, or part of the graph: `stg_customers+` takes everything downstream, `+prepared_orders` everything upstream.

Day 4: Working With AI Agents, Pipeline Design, and Airflow

DECODED

D4.1 DBT Debrief: What the Workflow Gives You

What have we learned about DBT? An experienced engineer would find these conspicuous *if missing*:

- If I can't test the thing rapidly as part of development, something's wrong
- If I can't test locally without interfering with anyone, something's wrong
- If I can't version and review my previous code, something's wrong
- And if I'm moving my hand with a mouse, something's wrong

With dbt, versioned files in Git, local testing, and Astronomer later today, we are applying the virtues of software engineering to data-product development. Fast iteration supplies evidence about what the computer does. One direct answer from the debrief: dbt is an OLAP tool, not an OLTP technology; if you're fitting it to transactional loads, change the tool.

D4.2 The Human Role When Working With AI Agents

Exploration vs. exploitation: are we too broad, or too narrow?

Heavy AI use over time can be addictive (intermittent rewards) and overloading (many assumptions made quickly). The remedies:

1. Reflect: are you getting good results *consistently*?
2. Simplify: reduce output; one thing at a time

AI results arrive on a variable-reward schedule, the same schedule that makes slot machines compulsive, so the pull to keep prompting is real. The human role is taste and judgement, including judgement about where things are complete: own the questions and the acceptance criteria, delegate the drafting.

D4.2a Challenges 4.1 & 4.2: Audit Generated Work

Challenge 4.1 (optional, pairs, ~30 min): review a plausible generated model against its STTM and against the agent's own summary; produce a defect list and a separate false-claims list. Do not repair the model.

Challenge 4.2 (individual, ~35 min): use a Catalyst DBT skill (`setup-dbt-project`, `write-dbt-model`, ...) to change the project, then use `git status` and `git diff` to find one changed detail the summary omitted.

The 4.1 model hides five defects: an inner join on the wrong key silently dropping rows, a filter deleting valid short positions, nulls vanishing in that same predicate, a GROUP BY that breaks the declared grain, and a summary false in checkable ways. Some a dbt test would catch; the filter, the join key and the false summary only a human reading against the STTM can. Git, not the agent's report, is the acceptance record.

D4.3 Demo: dbt snapshot and Slowly Changing Dimensions

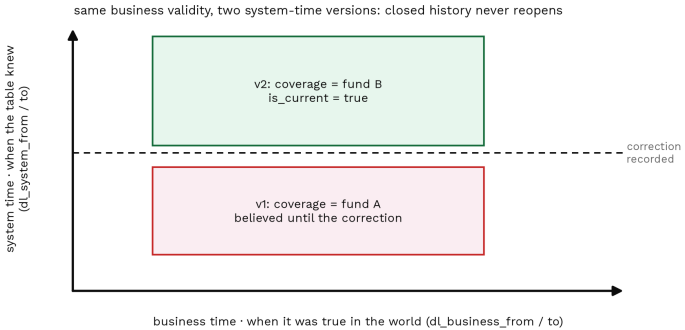
Portfolio-manager coverage is updated in place; overwriting answers *who runs the fund now* but loses *last quarter's attribution*.

SCD type	*Storage rule*	*Choose it for*
1	overwrite	current state only
2	add a date-ranged row	full point-in-time history
3	keep a previous-value column	current and one prior

`dbt snapshot` gives you Type 2: each version gains `dbt_valid_from` / `dbt_valid_to`; the current row has a null end.

Overwriting in place is SCD Type 1, and it answers only the first question. The snapshot's date-ranged rows are Type 2, and the next slide shows what happens when you add the second time axis.

D4.3a Bitemporal Time: Two Axes, Not One



A correction adds a new system-time version over the same business validity: what was believed stays queryable

Business time is when a fact was true in the world; system time is when the table recorded it. dbt has no built-in bitemporal mode; at CG the recorded pattern is the `bitemporal_load_v2` strategy with business, system, and is_current fields. Acceptance criteria: one current row per key, no overlapping windows, closed history never reopens.

Day 4 · Late Morning: Drawing Pipelines and Measuring Freshness

DECODED

D4.4 Challenge 4.3: Draw a Data Pipeline as a Flow

New mixed groups, about 50 minutes, on paper:

1. Choose one real pipeline and draw every source-to-output step
2. Mark what must happen first, what runs in parallel, what waits
3. Add the checks between steps
4. Show the routes for missing/late inputs, success, and failure

Output: a drawing another group could translate into task dependencies.

The diagram becomes the specification for an Airflow DAG, so ambiguous arrows become ambiguous schedules and controls. Real examples from the room: the positions integration landing by midnight with in-motion rules that page and at-rest rules that don't, and Morningstar ingestion contracts with completeness at 99.5%.

D4.5 Diagram Literacy: Three Kinds of Arrow

A pipeline diagram mixes at least three arrow types, and a rationalised diagram gives each visual type exactly one meaning:



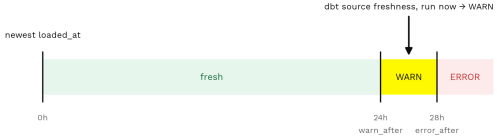
Three meanings, three line styles: if two arrows look the same, they must mean the same

Categorising every arrow in the source diagram exposed a real bug: the loop-back from the last step pointed at step one when it should return to the ingestion step. Diagrams become difficult exactly when each arrow silently means something different.

D4.6 Source Freshness as an SLI

The drawn promise “holdings arrive by 8 AM” is measurable in four lines of YAML on the source declaration:

```yaml
freshness:
  warn_after: {count: 24, period: hour}
  error_after: {count: 28, period: hour}
loaded_at_field: loaded_at
```



Airflow decides when things run; freshness measures whether the promise about the data held

dbt source freshness compares the newest loaded_at with the thresholds: the promise becomes a measurable window

The division of labour to keep straight for the afternoon: Airflow decides *when things run*; source freshness measures *whether the promise about the data held*. Airflow will happily start the run on time whether or not the file arrived; freshness, run as the first task, is how the pipeline notices the upstream promise broke before building anything on top of it.

D4.6a Demo: Freshness on the Real Sources

models/sources.yml

```yaml
# … (13 lines elided) …
sources:
  - name: custodian
    database: "{{ target.database }}"
    schema: "{{ target.schema }}"
    description: "Daily SFTP drop from the custodian: position-level holdings and cash."
    config:
      loaded_at_field: _extracted_at
      freshness:
        warn_after:  {count: 4, period: hour}
        error_after: {count: 6, period: hour}
    tables:
      - name: holdings_raw
        description: "Position-level holdings per fund per day."
# … (16 lines elided) …
  - name: bloomberg
    database: "{{ target.database }}"
    schema: "{{ target.schema }}"
    description: "Daily reference data + end-of-day prices."
    config:
      loaded_at_field: _extracted_at
      freshness:
        warn_after:  {count: 8, period: hour}
        error_after: {count: 24, period: hour}
    tables:
      - name: prices_raw
        description: "End-of-day close price per instrument per day."
# … (51 lines elided) …
```

The lab's sources.yml makes the promise measurable per feed: the custodian drop must be under 6 hours old, Bloomberg prices under 24

The four lines from the previous slide, in place: each source carries its own thresholds because each upstream makes a different promise. `dbt source freshness` reads `_extracted_at` and compares it against these, per source.

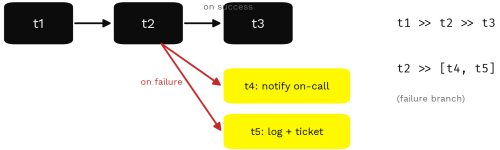
Day 4 · Afternoon: Airflow, Astronomer, and the Challenge Block



D4.7 How Does Airflow Work?

What problem does it solve? Orchestration: starting, stopping, timing tasks.

- DAG: the structured group of tasks, *plus its scheduling*
- Task: a programme; a group of operators
- Operator: the adaptor to another system (dbt, SQL, even Google Docs)
- Scheduler: runs the DAGs



The whiteboard DAG and the Python are the same drawing: a sequence on success, a fan-out on failure

Airflow stole the term DAG: everything is a directed acyclic graph, dbt's model tree included, but in data engineering "DAG" now just means an Airflow task sequence. In practice, using Airflow comes down to understanding your operators; the rest is rote configuration you look up.

D4.8 Astronomer and Cosmos

Astronomer is managed Airflow, twice over:

1. a local docker wrapper around Airflow: `astro dev start` (dev)
2. a remote cloud “managed” version, with the Astronomer UI (live)

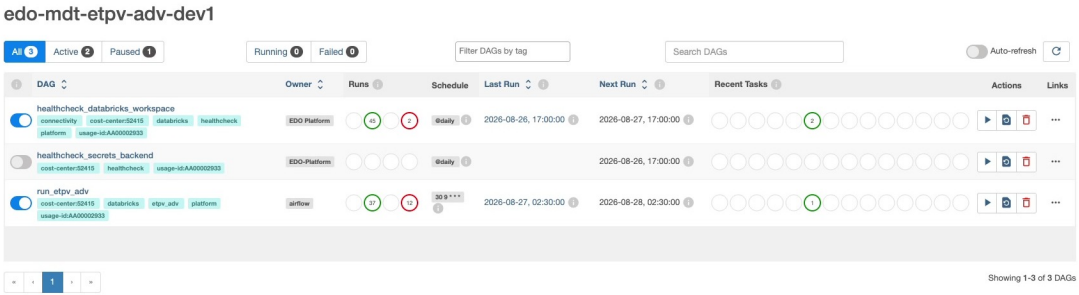
Cosmos provides Airflow operators for dbt:

- `DbtDag`: run a whole existing dbt project, no further configuration
- `DbtTaskGroup`: mix dbt steps with other kinds of task

If all you want is “7am daily, run my dbt project, create all the models”, `DbtDag` is the entire file. Behind the brand names: Airflow is mostly a Python wrapper around cron, and “managed” means a middleman runs it for us.

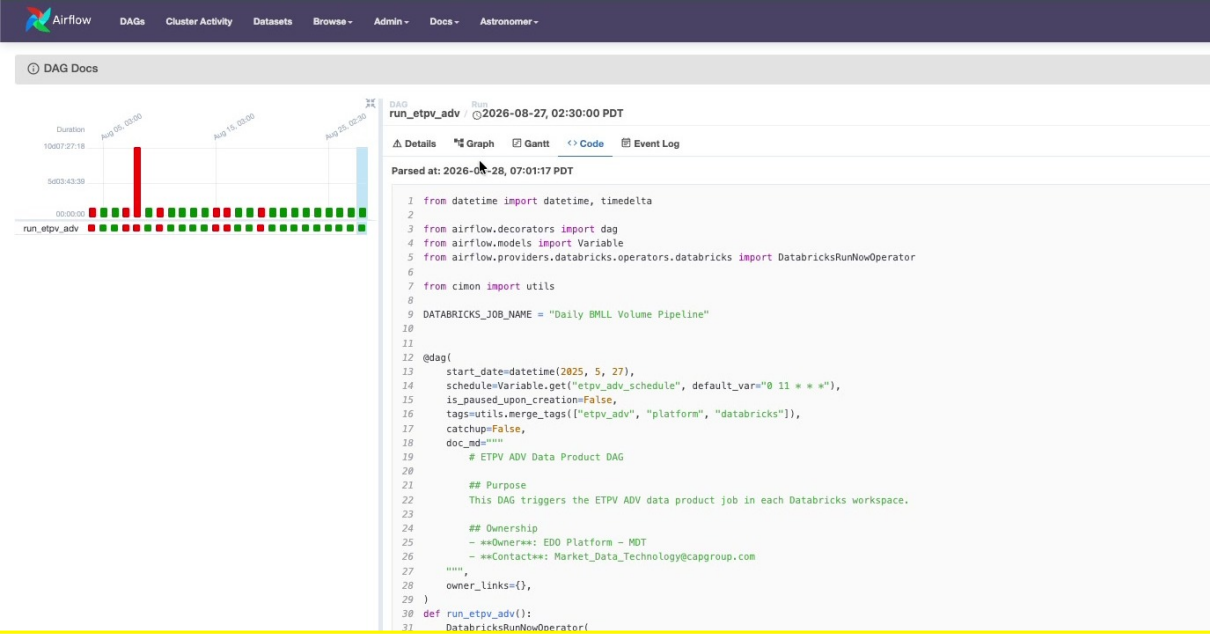
D4.9 The Airflow UI

From the web interface you can manually trigger DAGs, inspect logs, and monitor task status, plus the graph and Gantt views of every run.



Every DAG carries an owner and a schedule, and platform healthchecks run beside the products: operations reads this table, not your code

Failed tasks show red and can be retried from the UI. Airflow is a code-first, Python-first system, though there's more YAML-based configuration at CG than you might expect.



D4.10 Demo: A Production DAG, In Full

The whole file for a shipped product: one DatabricksRunNowOperator call, a schedule read from an Airflow Variable, ownership in doc_md. Everything you write in the challenges is here in the same shape, and nothing more: the work happens in Databricks, and Airflow triggers it, times it, and owns the record.

D4.11 Sequencing Tasks

```
# t2 depends on t1 running successfully — equivalents:
t1.set_downstream(t2)
t2.set_upstream(t1)
t1 >> t2
t2 << t1

# Chaining and fan-out:
t1 >> t2 >> t3
t1 >> [t2, t3]          # a list can be a dependency set
[t2, t3] << t1
```

Read >> as “and then”: do this, and then do that; do this, and then all of these. The arrows in your pipeline drawing from this morning *are* these symbols in the Python.

D4.12 Demo: The DAG Definition

```python
# dags/portfolio_analytics_daily.py (the DAG call; imports elided)
with DAG(
    dag_id="portfolio_analytics_daily",
    description=(
        "Daily refresh of the Client Portfolio Analytics product on "
        "Databricks. SLA: mart published by 07:30 UK from a 06:00 UK "
        "weekday schedule."
    ),
    default_args=default_args,   # owner, retries=2, failure callbacks
    start_date=pendulum.datetime(2026, 5, 1, tz="Europe/London"),
    schedule="0 6 * * 1-5",
    catchup=False,
    max_active_runs=1,
    sla_miss_callback=on_sla_miss_callback,
    tags=["dbt", "portfolio-analytics", "cosmos", "databricks",
          "production", "usage-id:AA00002933", "cost-center:52415"],
    doc_md=__doc__,
) as dag:
```

Everything operational is declared rather than coded, and the description states the SLA in plain English: the promises from Day 1 have become configuration. The tags carry the required usage-id and cost-center.

D4.12a Demo: The Tasks and the Wiring

```
start = EmptyOperator(task_id="start")
source_freshness_check = PythonOperator(
    task_id="source_freshness_check",
    python_callable=_check_source_freshness,
)
dbt_pipeline = DbtTaskGroup(
    group_id="dbt_pipeline",
    project_config=ProjectConfig(DBT_PROJECT_DIR.as_posix()),
    profile_config=profile_config,   # execution + render config elided
)
publish_to_catalog = PythonOperator(
    task_id="publish_to_catalog",
    python_callable=_publish_to_catalog,
    sla=timedelta(minutes=90),
    on_success_callback=on_success_callback,
)
end = EmptyOperator(task_id="end")

start >> source_freshness_check >> dbt_pipeline >> publish_to_catalog >> end
```

The freshness gate runs first so a stale feed fails before any compute is spent; publish runs last, carrying the 90-minute SLA. `DbtTaskGroup` points at the dbt project you already have; no dbt logic is repeated in Airflow.

D4.13 Challenge 4.4: Orchestrate the DBT Pipeline with Airflow

The `03-airflow-orchestration` route: ten guided sections, then ten production-operations challenges (~90 + ~100 minutes).

- Build `start` → `source_freshness_check` → `dbt_pipeline` → `publish_to_catalog` → `end`
- `PythonOperator` for Python, Cosmos `DbtTaskGroup` for dbt; no **`BashOperator`** / **`SSHOperator`** (governance blocks arbitrary shell)
- Add the required **`usage-id`** and **`cost-center`** tags

Static verification parses the Python and checks all four edges of the five-node chain; the Astro run supplies execution evidence: graph, logs, schedule, retries, alerts, rerun behaviour. The closing exercise generates a DAG with Catalyst (`generate-dag`) and audits it against your drawn SLA.

D4.14 The Paved-Path Scheduling Recipe

```
positions-scheduling/
├── Dockerfile          # FROM an Astro Runtime image
├── dags/
│   └── positions_dbt.py
├── dbt/
│   └── positions/      # the Day-3 dbt project, vendored in
└── requirements.txt    # astronomer-cosmos, dbt-databricks
```

The whole week in one repo layout: Docker packages it, the DAG schedules it, the dbt project transforms, and requirements.txt pins the libraries that join them. Day 5 shows this shape deployed for real, and the real repos look very like it, which is not a coincidence.

D4.14a Stretch: DBT as a Databricks Job

dags/databricks_submit_run_dag.py

```python
# … (63 lines elided) …
with DAG(
    dag_id="portfolio_analytics_submit_run",
    description=(
        "Paved-path variant: dbt runs inside Databricks as a Job. "
        "DatabricksSubmitRunOperator submits each step and polls."
    ),
    default_args=default_args,
    start_date=pendulum.datetime(2026, 5, 1, tz="Europe/London"),
    schedule="0 6 * * 1-5",
    catchup=False,
    max_active_runs=1,
    sla_miss_callback=on_sla_miss_callback,
    tags=["dbt", "portfolio-analytics", "databricks", "submit-run", "production", "usage-id:AA0002933", "cost-center:52415"],
    doc_md=__doc__,
) as dag:

    # Step 1 — source freshness. The notebook runs `dbt source freshness` and
    # exits non-zero if the custodian feed breaches error_after, failing the
    # task and short-circuiting the chain.
    check_source_freshness = DatabricksSubmitRunOperator(
        task_id="check_source_freshness",
        databricks_conn_id=DATABRICKS_CONN_ID,
        json={
            "run_name": "portfolio_analytics_source_freshness",
            "existing_cluster_id": DATABRICKS_EXISTING_CLUSTER_ID,
            "notebook_task": {
                "notebook_path": f"{DBT_NOTEBOOK_BASE}/check_source_freshness",
            },
        },
        polling_period_seconds=30,
        timeout_seconds=900,
    )
# … (49 lines elided) …
    check_source_freshness >> run_dbt_models >> test_dbt_models >> generate_dbt_docs
```

The paved-path variant: Airflow submits each dbt step to Databricks and polls. The DAG keeps the schedule, the SLA and the record; the compute and logs live with the data

The stretch DAG from the challenge repo, and the CG paved path: dbt runs inside Databricks as a Job, so the Airflow worker only orchestrates. The trade-off against Cosmos is granularity: one opaque job task, not one task per model. Note the same governed tags: usage-id and cost-center.

D4.15 Day 4 Recap

- The software-engineering virtues now apply to your data product
- Snapshots keep history; bitemporal adds the second time axis
- A pipeline drawing is a DAG specification; one meaning per arrow
- Freshness is an SLI: four lines of YAML measure the arrival promise
- Airflow: operators are the adaptors; Cosmos is the dbt adaptor

Tomorrow: data operations, and the end-to-end picture.

The Airflow material is a little harder than the DBT material: there's logging in there, and success and failure handling. That's deliberate; the production-operations challenges are where the operating contract becomes real.

Day 5: Infrastructure, Data Operations, and the End-to-End Challenge

DECODED

D5.1 Purpose of the Day, and the Poll

Hands up: I have seen or used the Harness interface · I have followed a real change through CI/CD into a released environment.

In the Day-1 asset view, a data product is code + infrastructure + data + promises. Four days covered data, promises and code; today adds the infrastructure, under the umbrella of data operations.

You will rarely configure this layer directly, but you need it to interpret the paved path end to end. The practical goal: produce a data product *with Catalyst* while understanding the generated work, because people use these products to make consequential decisions.

D5.2 Worked Production Example: Two Paved-Path Products

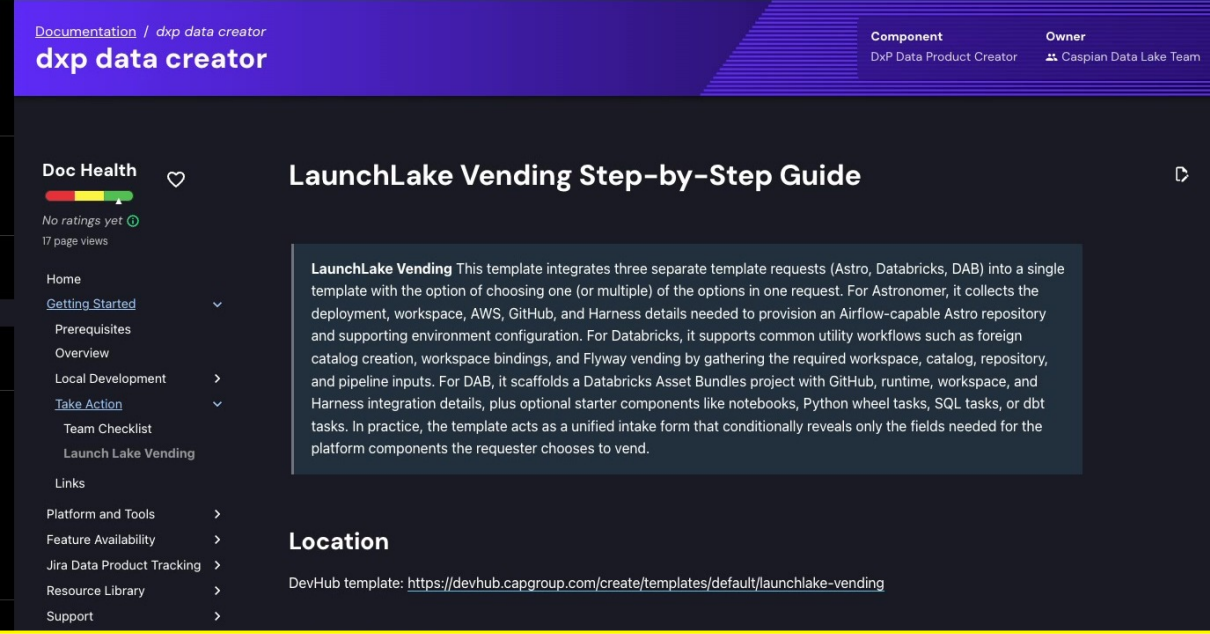
A production team runs *Average Advisor* and *Next-Gen Portfolio Construction* on the paved path. The next slides show the real screens along their route:



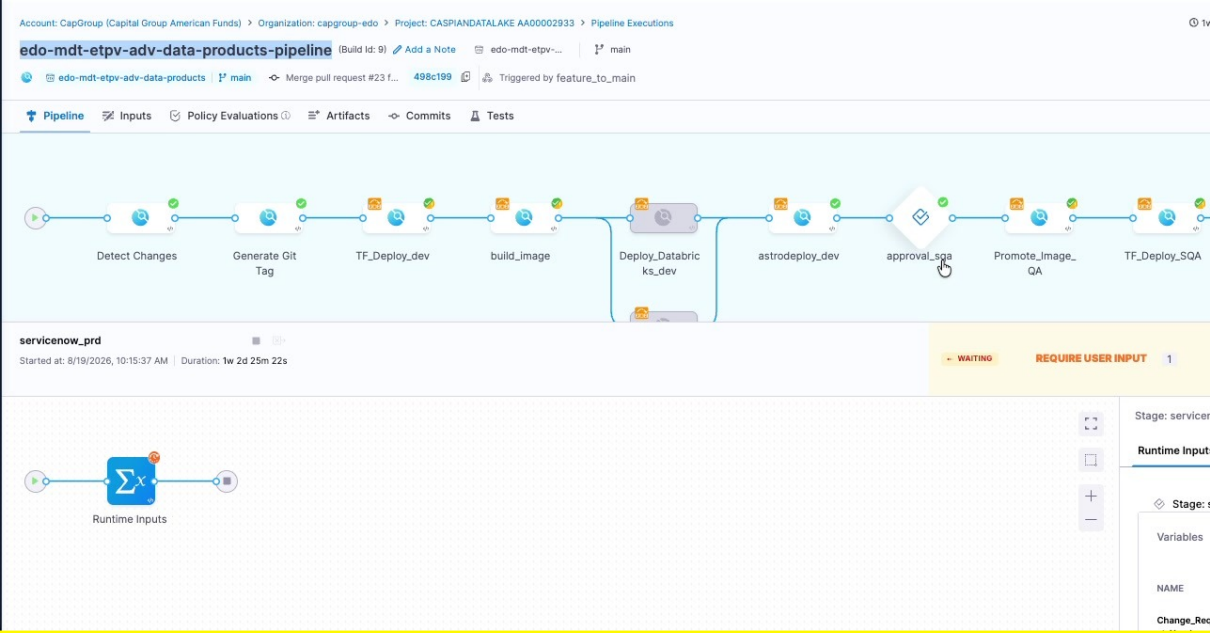
the life of a paved-path data product, birth to operations

The same tools you used all week, plus the approvals and the observers around them

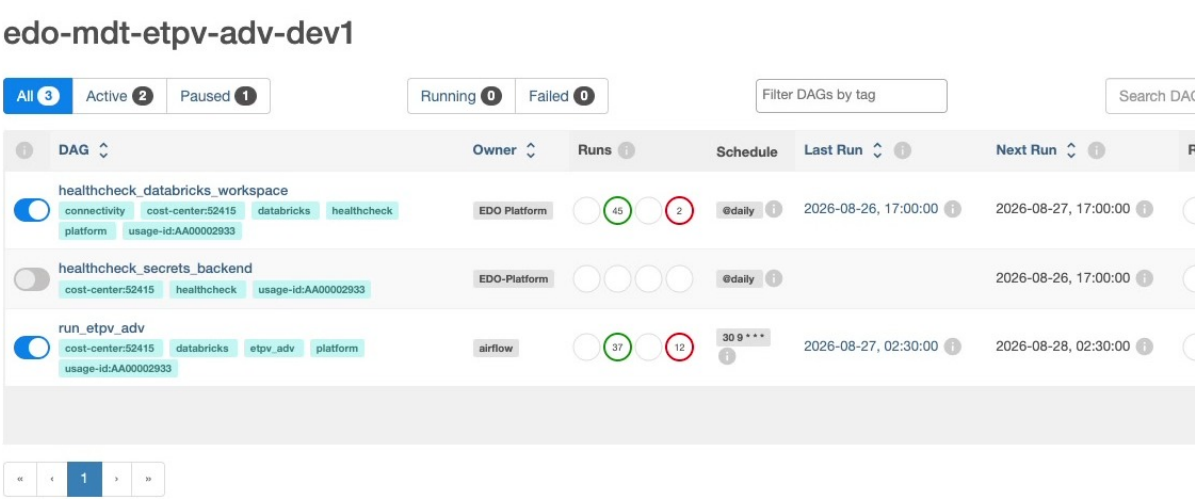
The team's account: ~15 raw Pecan files and ~15 FactSet files land in staging, integration merges them, prepared (curated) is the consumer-facing zone, and consumers mostly reach the data through an API service. The core process has been stable for roughly nine years: manage the code, deploy it, monitor quality, provide access. The tools change; the responsibilities don't.



D5.2a Vend: A Product is Born on DevHub
The vending template is meant to be end to end: with your ADM ID it provisions the repo, the Harness pipeline, the Astro workspace, and the Databricks connection in one intake form. The repos it creates look very like the dev container layout, which is not a coincidence.

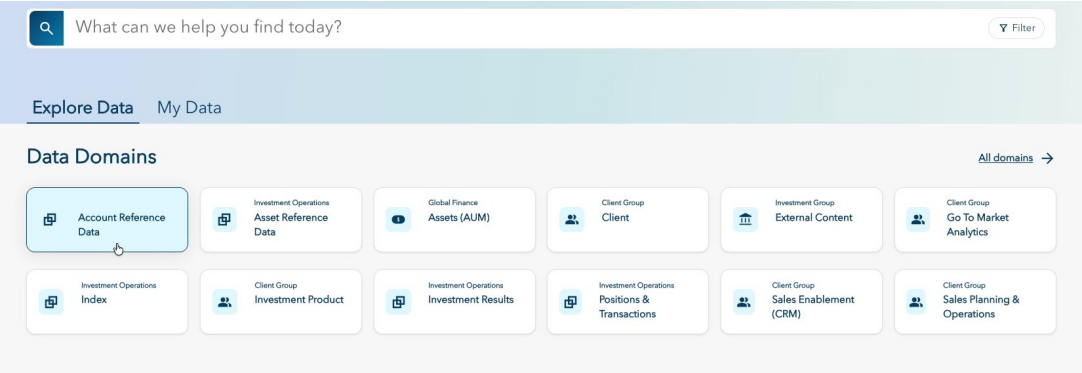


D5.2b Pipeline: Harness Runs Every Merge
The reviewers approve the PR; the merge fires a webhook; the pipeline runs. At the production gate it stops and waits for an approved ServiceNow change request, best thought of as the legal and audit record rather than part of CI/CD. Teams typically restrict the production pipeline to a few dedicated people.



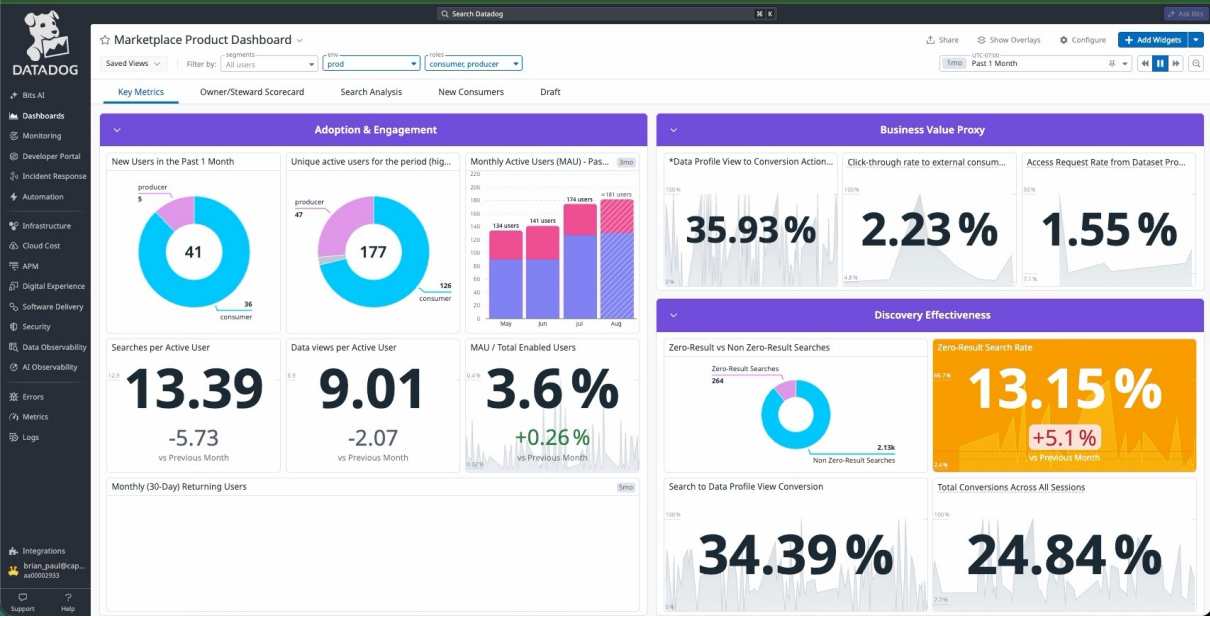
D5.2c Deploy: Your Astro Deployment
Why schedule with Airflow rather than Databricks jobs? Standards: the data operations group keeps eyes-on-glass across the important DAGs, and bare Databricks jobs would be invisible to that view. A production DAG can be one step calling a notebook; the orchestration layer is the point.

D5.2d Publish: The Marketplace

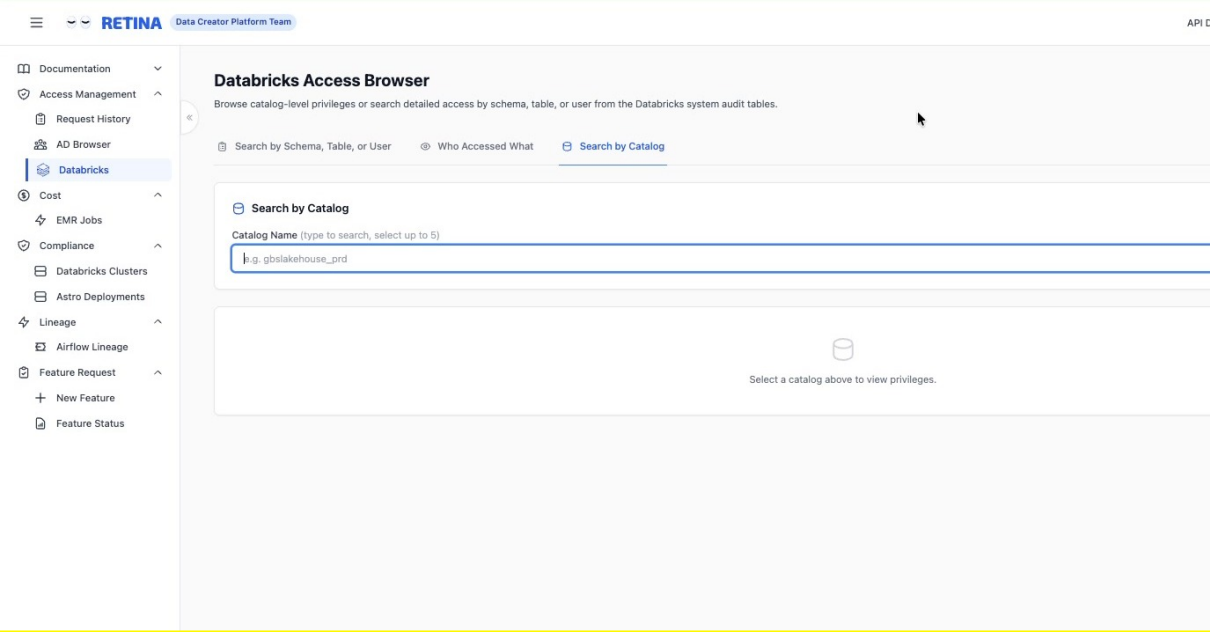


The twelve data domains: where consumers find your product, and where you check whether it already exists

Registering your product here is part of shipping it: data meant to be shared should be discoverable, otherwise we're just creating silos. Catalyst's discover phase searches the marketplace and Unity Catalog and asks whether something similar already exists: enhance rather than rebuild.

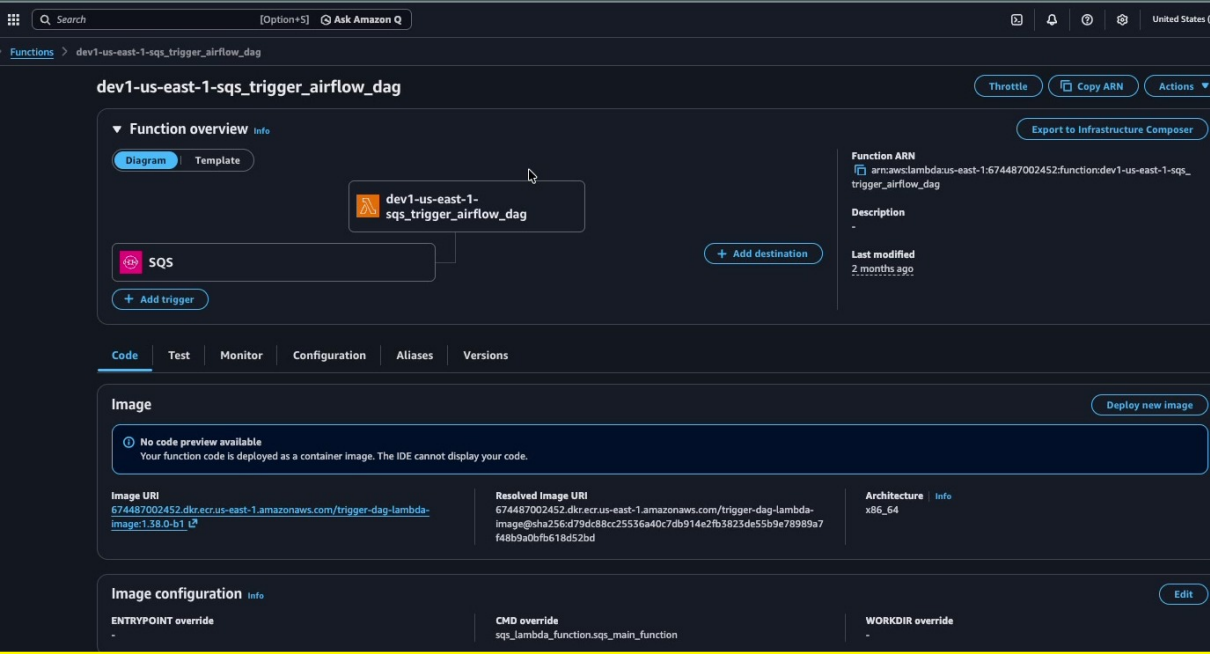


D5.2e Observe: Datadog
Monitors watch conditions per DAG (did the DQ check pass, did it run in its window), and for the important pipelines the data operations folks triage from exactly this view. This is where your SLIs actually live.

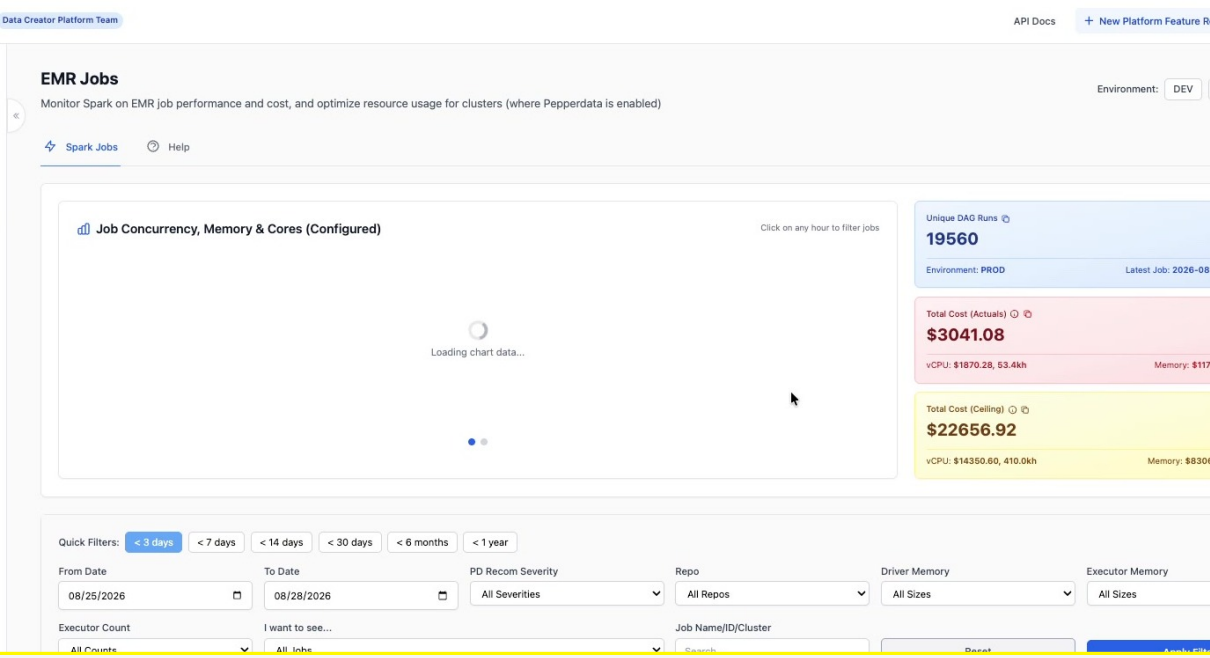


D5.2f Govern: Who Can Read What

Search access by catalog, schema, table or user, and see who accessed what. An open design question at CG: should everyone be able to *browse* schemas (not the data) so discovery works? And a practical gotcha: a catalogue must be *bound* to your workspace before you can use it.



D5.2g Event-Driven: S3 → SQS → Lambda → Airflow
Not everything runs on a clock. A vendor (say Morningstar) drops a file into S3; S3 notifies SQS; SQS triggers this Lambda; the Lambda triggers the Airflow DAG; the DAG runs dbt. The whole paved path, fired by a file arriving.



D5.2h Aside: What Does It Cost?
Optional FinOps stop: job concurrency, actual cost against ceiling, and tuning recommendations, per pipeline. Cost tracking views like this are expected to grow.

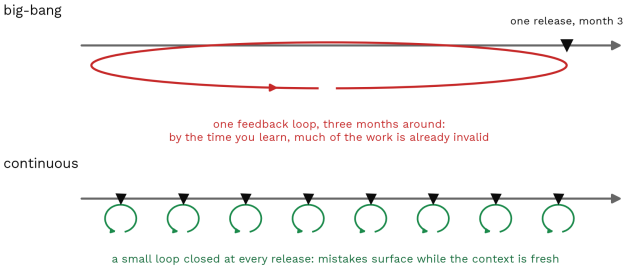
Day 5 · From Commit to Cloud: Releases, Containers and Orchestration



D5.3 How Does a Data Product Get Released?

Continuous Integration (*“merging”*) · Continuous Deployment

- *continuous* = as-and-when the quality checks pass, usually daily or weekly



The cadence is the feedback loop: months of drift, or days

The three-month cycle genuinely works for one to three developers. In larger teams, by the time you release, other teams and their understanding have moved, and some of what you built is no longer valid. Releasing often keeps the shared understanding current, so mistakes surface fast.

D5.4 GitOps, and the Role of Harness

- PR = permission to integrate one feature branch into main
- GitOps: *when code is integrated, code is deployed*
- Harness glues the GitHub events to the deployment events

Tie the live environment to the Git history and GitHub becomes the record of your live system: the history of your code is the history of your deployments. CG turns the GitOps dial about halfway, keeping the ServiceNow approval step in between, because full GitOps would release without approval.

D5.5 Challenge 5.1: Approve or Request Changes?

You are the reviewer. The contract says: `exposure_us` is a float, 0 to 1; nulls must fail the pipeline. The PR diff:

```
 models:
   - name: stg_morningstar_exposures
     columns:
       - name: exposure_us
         tests:
-          - not_null
```

Approve, or request changes? Defend it. (10 minutes, groups of three)

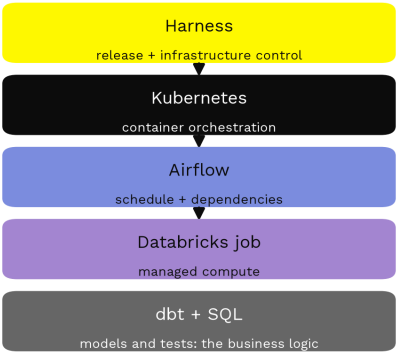
The PR is where acceptance happens: every promise in the contract is either preserved or broken by what merges, and this is the last screen where anyone gets to say no. Sitting on this gate doesn't require writing the change; knowing the contract is the qualification, and that belongs to the data creator. A deleted test is a block of red lines, easy to skim past.

D5.6 The Cloud, and CG's Environments

- IaaS is what matters here: renting machines (EC2 = "Amazon VM", S3 = "Amazon files and folders")
- pricing intuition: a continuously running service $\approx$ 750 hours/month
- you rent by writing code, not clicking consoles
- and everything exists three times: dev $\rightarrow$ QA (SQA) $\rightarrow$ prod

The environments are about *software*, not data: how a change is tested before it reaches the infrastructure the promises depend on. Translate every vendor acronym back to the plain thing being sold, and multiply the hourly rate by 750 before calling it cheap.

D5.7 The Layers of Orchestration



Each layer localises one concern; the stack narrows from infrastructure to the business logic

Why do orchestrators keep appearing? Whenever engineers do something repeatedly, an orchestrator appears to do it for them: Airflow “types out” run-this-dbt-job; Kubernetes “types out” run-this-container. Harness drives the outermost layer; dbt is the innermost, a row-and-column orchestrator.

D5.8 Challenge 5.2: The 3 a.m. Failure Drill

The nightly DAG failed a dependency check; the contract promises data by 7:30. Trace the failure backwards through the stack (20 to 30 minutes, groups, on paper):

- Airflow task logs: did it run, and which task failed?
- Databricks job/query history: did the SQL fail, on which query?
- Harness execution history: did someone deploy last night?
- Datadog: which monitor shows the breach, and who was alerted?

Then draft the duty-to-notify message.

A failure the morning after a release points first at the release. The message states who is told, by when, what is known, what remains unknown, and when the next update comes. Data creators often read these systems rather than operate them; the drill connects an SLA promise to the evidence.

D5.9 Harness, Indirection, and Conway's Law

- This layering is the engineering technique of indirection: each layer localises one concern
- Conway's Law: systems evolve to match the communication structure of the business; *tell me your teams and I'll tell you your architecture*
- It also explains the folders: earlier database-to-database integrations made replacement disruptive, so teams decoupled by transferring files

The 500-line thought experiment: the core operation could fit in one Python file (download code, schedule it, run it, check the result). The extra complexity comes from cloud infrastructure and from managing separate concerns through different interfaces and teams. Inherited architecture is Los Angeles: add the metro line; don't delete the city.

Day 5 · The Storage Story: Lakehouse Theory to Mini-Lakehouse Practice

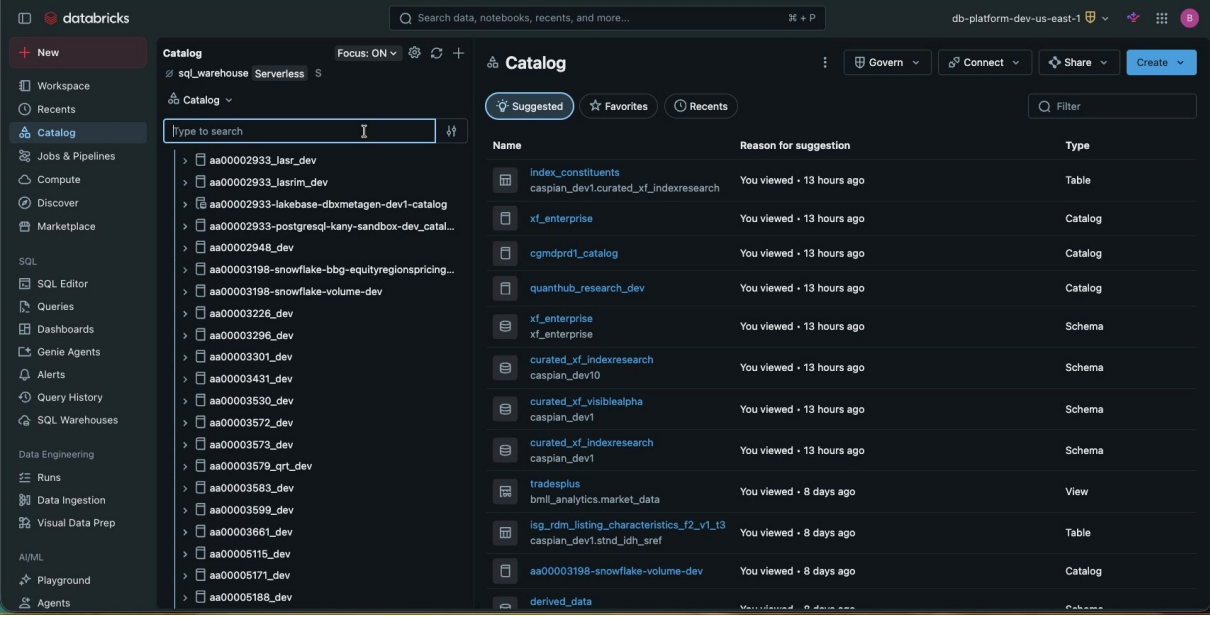


D5.10 Caspian: the Lakehouse in Practice

Caspian is CG's lakehouse: files and folders in S3, holding Delta-format tables, with metadata so we can manage them, queried through Databricks.

To the reader it looks like relational tables: catalog → schema → table. Underneath, it is open files in S3. The next slide is the real catalog.

Unity Catalog is where the governance lands: grants, plus column masks and row filters evaluated in the engine whatever route the query arrives by. Per-table functions don't scale to hundreds of products, so governed tags carry the policy: tag a column PII and the control follows the tag.



D5.10a The Catalog, Up Close
The tree on the left is the governed namespace: one catalog per product family, schemas inside, tables inside those. Every aa-prefixed entry is a real product's dev catalog, and the Suggested pane is the same discovery surface Catalyst searches before recommending you build anything new.

D5.11 Data Mesh, Briefly

Data mesh is the opposite of a centralised data team: product creators spread throughout the business, creating and self-serving their own products.

It is an *organisational* concept, not a technology. The combination: mesh (people) + lakehouse (infrastructure) + dbt (code) + SLAs (promises).

The team closest to the work is usually best placed to understand the data, its consumers, and the promises the product must keep. How CG balances local creation with central platform and governance is CG policy; treat the course description as a working model and confirm current guidance.

D5.12 Challenge 5.3: Design the Harness Pipeline

Groups, on paper, 30 to 40 minutes. Sketch the release pipeline for the mini-lakehouse product:

1. The event that triggers a release
2. The stages: provision infrastructure · deploy data jobs · validate
3. dev, SQA, prod, and the promotion route between them
4. Each approval gate and the role that holds it

Use yesterday's three arrow meanings: trigger, stage order, approval. Debrief against the repository's actual Harness configuration, whose stages are exactly provision-infrastructure, deploy-data-jobs, validate, triggered by a merge to the live branch.

D5.13 Walkthrough: The Mini Lakehouse

A self-contained demonstration of the paved path working together:

- Airflow: check sources → load bronze → build silver → publish gold → Unity Catalog
- bronze: raw CSVs (clients, positions, prices) + timestamps
- silver: cleaning; gold: the big join, a portfolio exposure table
- Harness: provision → deploy → validate; one S3 bucket per layer

Faithful in shape, not in scale or hardening: MinIO stands in for S3, one Spark server for governed compute, shared lab credentials for SSO. One small edit proves the contract is active: make a source quantity negative and the staging rule drops the row, the curated result falls from four rows to three, and the row-count assertion fails.

D5.14 Recording Decisions: the RFD (Challenge 5.4)

An RFD needs three sections: Context (constraints and a real alternative), Decision, Consequences (including at least one negative, with its mitigation).

Challenge 5.4: write one from your build, until a partner could implement the decision without asking what you meant.

When reviewing an RFD, a Consequences section containing only benefits is grounds for requesting changes: the author must name what the team gives up and which mitigation makes that cost acceptable. Otherwise a later reader cannot distinguish an evaluated decision from a preferred option written after the fact.

Day 5 · The End-to-End Challenge and Mop-Up

DECODED

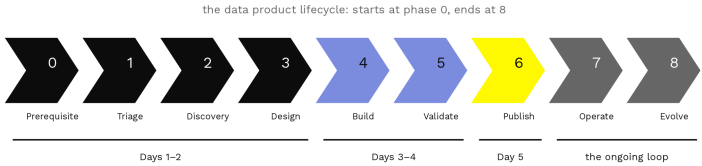
D5.15 Challenge 5.5 (Optional): A Data Product, End to End

Groups of two or three, about one hour:

1. Choose a non-sensitive or simulated product (or a numbered case study)
2. Use Catalyst for the contract, mapping, dbt models, tests, seeds
3. Commit as you go; open and review a PR if time permits
4. Run the pipeline in Databricks; confirm the row counts match via the `databricks` CLI

Airflow is optional here because the main job for many data creators is dbt: the contract, the SLAs and therefore the tests, and the mapping all live in that layer. Moving from dbt to Airflow to Harness is a slide toward an engineering-intensive skill set most creators won't directly own. Each group reports for three minutes: what was the problem, how did Catalyst help, did it work?

D5.16 Catalyst Mop-Up: Nine Phases, Four Roles



Nine phases, numbered 0 to 8, and where this course sits on them: design by Day 2, build and validate by Day 4, publish on Day 5

Phase 0 names four roles: data owner (sponsor/approver) · data product manager (requirements) · data steward (quality) · data product creator (you).

Trick question: count the poster and you find seven phases, but it starts at phase 0 and ends at 8. Mapped onto this course: days 1 and 2 reach design, days 3 and 4 are build and validate, day 5 talks around publish; operate and evolve are the ongoing loop. Catalyst's Airflow skill is `generate-dag`.

D5.17 After the Course: the Capstone and Enablement

Where offered, the capstone is a full day: a required guided Phase 1 build of a data product on the paved path, then optional Phase 2 challenges for those who finish.

After that: the enablement phase: system access, team integration, Confluence guidance, and Catalyst evolving.

The capstone certifies, in the organisation's language, that you have created a data product. It doesn't mean you can roam AWS independently; it means the minimum capability is confirmed, observed.

D5.18 Course Recap: The Week on One Slide

- A data product is data + promises: SLAs, ownership, governance
- Design outward: consumers, canvases, empathy, stakeholders
- Build on the paved path: GitHub → Docker → dbt → Airflow → Databricks
- Test as you go; promise only what you measure; record decisions
- Use AI with the human owning the questions, and verify the files

The paved path is a folder somewhere that something copies files into and out of, changing their alignment along the way, made reliable, permissioned and repeatable. If you can draw that figure and say who each promise on it is made to, the week did its job. See you in the Teams channel.

Day 6: The Capstone Event

DECODED

Data Creators Capstone Event

Get your name cards from the table on the far side of the room, then sit anywhere you like. We'll start at 9.05am.

- The day: 9am to 2.30pm work · 3pm presentations · 4pm finish · lunch 12 to 1pm · check-ins at 10.30am and 1.30pm
- First: do a *fresh* `git clone` and start a NEW dev container ASAP!
  `https://github.com/cg-open-itg/edo-training-data-products`
- Working: you may work together, and use Catalyst and AI to support. Each person should have a solution on their own laptop.
- Blocked? Add blocking questions to the cohort's Teams group, especially for platform support.
- Enjoy your free embroidered DXP cap; other merch prizes may be awarded!

Phase 1 shouldn't take too long, and you can extend the team phase if you aren't finished. Phase 1 is the baseline completion for the capstone, but please move on to Phase 2, especially if you're a technical learner. The lunch code and the cohort's Teams channel are shared on the day.